Project Title
OnChord Interim Report

TU857
BSc in Computer Science (infrastructure)

Student Name: Emmanuel Awode
Student Number: C22320583

Supervisor: Dr Fatmaelzahraa Eltaher

School of Computer Science
Technological University, Dublin

Date: 23/11/2025

Abstract

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Student Name: Emmanuel Awode C22320583

Date: 23/11/2025

Acknowledgements

I would like to begin by sincerely thanking my supervisor, **Fatmaelzahraa Eltaher**, for her guidance, patience, and constant support throughout this project. She played a huge role in helping me understand the machine learning concepts behind my work and in shaping the project into something realistic and achievable. Her feedback and reassurance at each stage made the technical side of this project far less intimidating.

I would also like to thank the lecturers in the **School of Computer Science at Technological University Dublin** for their teaching and support over the course of my degree. Their willingness to answer questions, explain difficult topics, and share their experience has been essential to my learning and to the completion of this project.

To my classmates and peers, thank you for listening to my ideas, giving feedback, and encouraging me to follow the musical direction of this project, knowing how important music is to me. Your interest and positivity helped me stay motivated and confident in the theme I chose.

Finally, I am deeply grateful to my **friends and family** for their constant encouragement, understanding, and belief in me. Your support throughout the stressful moments, and your patience while I focused on this work, have meant a great deal and have helped me reach this point.

Contents

1. Introduction 1
1.1 Project Background 1
1.2 Project Description 2
1.3 Project Aims and Objectives 2
1.4 Project Scope 4
1.5 Thesis Roadmap 5
2. Literature Review 7
2.1 Introduction 7
2.2 Alternative Existing Solutions 7
2.3 Technologies Researched 8
2.4 Other Research 10
2.5 Existing Final Year Projects 11
2.6 Conclusions 11
3. System Analysis 12
3.1 System Overview 12
3.2 Requirements Gathering 12
3.3 Requirements Analysis 18
3.3.1 Actors and System Boundaries 19
3.3.2 Core System Processes 19
3.3.3 Data Entities and Relationships 20
3.3.4 System Architecture Model (Conceptual) 20
3.3.5 Mapping Requirements to the System Model 21
3.3.6 Initial System Model Summary 22
3.4 Initial System Specification 22
3.4.1 Functional Requirements (FRs) 24
3.4.2 Non-Functional Requirements (NFRs) 25
3.4.3 Appropriate System Architecture 25
3.5 Conclusions 27
4. System Design 28
4.1 Introduction 28
4.2 Software Methodology 28
4.3 Overview of System 28
4.3.1 Logical Architecture Diagram 29
4.3.2 Physical Architecture Diagram 32
4.3.3 Deployment Diagram 34
4.4 Design System 35

4.4.1 User Interface & Experience Design (UI/UX) 35
4.4.2 User Interface & Experience Design (UI/UX) 36
4.4.3 User Interface & Experience Design (UI/UX) 37
4.4.4 Data Flow & Interaction Design 37
4.4.5 Machine Learning Integration Design 38
4.4.6 Database Design (Supabase / PostgreSQL) 38
4.4.7 Summary 39
4.5 Database Design - ERD Explanation 40
4.6 Conclusions 43
5. Testing and Evaluation 44
5.1 Introduction 44
5.2 Plan for Testing 44
5.2.1 Unit Testing 44
5.2.2 Integration Testing 45
5.2.3 System Testing 45
5.2.4 User Acceptance Testing (UAT) 46
5.2.5 Structured Test Plan Table 46
5.3 Plan for Evaluation 46
5.3.1 Usability Evaluation 46
5.3.2 Performance Evaluation 47
5.3.3 Reliability & Error Handling 47
5.3.4 Machine Learning Evaluation 47
5.3.5 Requirements Evaluation 48
5.4 Conclusions 48
6. System Prototype 49
6.1 Introduction 49
6.2 Prototype Development 49
6.2.1 Authentication and Landing Screen 50
6.2.2 Home Feed and Recent Reviews 51
6.2.3 “Your Space” - Profile and Stats 52
6.2.4 Concerts & Events 53
6.2.5 Messaging Interface 53
6.2.6 Review Calendar View 54
6.2.7 Music Insights - Taste Match 55
6.2.8 Music Insights - Mood Analysis 56
6.2.9 Discover & Notifications 57
6.3 Results 58

6.4 Evaluation 59
6.4.1 Usability & UX 59
6.4.2 Performance 59
6.4.3 Reliability & Error Handling 59
6.4.4 Machine Learning Behaviour 59
6.4.5 Requirements Check 59
6.5 Conclusions 59
7. Issues and Future Work 61
7.1 Introduction 61
7.2 Issues and Risks 61
7.2.1 Integration with External APIs 61
7.2.2 Machine Learning Quality and Scope 61
7.2.3 Time and Complexity 61
7.2.4 Performance and Reliability 62
7.2.5 UX and Accessibility 62
7.2.6 Conclusion 62
7.3 Plans and Future Work 62
7.3.1 Prioritise Core Features 63
7.3.2 Phase the Machine Learning Work 63
7.3.3 Handle API Risks 63
7.3.4 Improve UX Iteratively 63
7.3.5 Documentation and Reflection 64
7.3.1 Project Plan with GANTT Chart 64
References 66
Appendix A: System Model and Analysis 1
A.1 Overview 1
A.2 Requirements Gathering Methods 1
A.3 Requirements Analysis Approach 5
A.4 System Model Artefacts 5
A.5 Summary 6
Appendix B: Design 1
B.1 Design Approach 1
B.2 Frontend Design (React + TypeScript) 1
B.3 Backend Design (Node.js + Express) 1
B.4 Data Model (Supabase / PostgreSQL) 2
B.5 Machine Learning Microservice (FastAPI + Python) 2
B.6 Summary 2

Appendix C: Prompts Used with ChatGPT 1
C.1 Project Idea and Scope 1
C.2 Name, Branding and Colour Scheme 1
C.3 Requirements, Use Cases and System Model 1
C.4 Architecture, ERD and Diagrams 1
C.5 Report Writing and Structure 1
Appendix D: Additional Code Samples 1
D.1 Session + Onboarding Management (Auth + localStorage) 1
D.2 Theme & Accessibility - Dark Mode + Accent Colour 3
D.3 Review Creation and Editing Flow 4
Appendix E: Additional project analysis 1
Appendix F: Technology Research References 2
Appendix G: Other Use Case Narratives 3

1. Introduction

1.1 Project Background

Music consumption continues to grow globally, with streaming now dominating how listeners access and explore new content. According to the 2024 IFPI Global Music Report, worldwide recorded music revenues grew by 4.8% in 2024, reaching $29.6 billion, with paid streaming subscriptions surpassing 752 million users (Music Business Worldwide, 2025).

This shift toward streaming has changed not only how people consume music but how they engage with it socially. Research from the World Economic Forum (2023) highlights that while streaming has made music more accessible than ever, it has also reduced traditional social elements of music discovery such as sharing physical albums, attending listening parties, or discussing new releases in person.

Similarly, industry commentary from BBC Tech (2023) argues that the algorithm-driven nature of modern streaming can make the listening experience feel increasingly individualised and less communal, creating a growing need for platforms that restore the social and expressive side of music culture.

At the same time, platforms built for other media such as Letterboxd for film and Goodreads for books have shown that users enjoy journaling, sharing reviews, and expressing their tastes in a social, community-driven setting. Forbes (2023) described Letterboxd as thriving because it "embraces niche expression and community identity", highlighting a wider shift towards platforms that support identity-driven media engagement.

However, existing music platforms such as Last.fm or Stats.fm remain largely analytics-focused, offering limited opportunities for social interaction or personalised expression beyond statistics. Music recommendation algorithms themselves are often criticised for lacking transparency and significance; for example, The Guardian reported concerns about the reliability and authenticity of algorithmic music data, especially as AI-generated content becomes more common lately (The Guardian, 2025).

Academic research further supports the importance of social and contextual factors in music consumption. A 2024 study published on arXiv found that social media presence significantly influences music demand and listener behaviour, recreating the idea that music engagement is social and expressive instead of being solely algorithmic.

These industry trends, media reports, and academic findings collectively show a gap in digital music platforms: listeners lack a dedicated space where they can share opinions, express identity, document their listening journeys, and connect with others who share similar tastes.

As both a music producer and software developer, this gap directly inspired the creation of OnChord. The platform aims to blend music journaling, personalised insights, and community engagement in a single space.

1.2 Project Description

OnChord is a web-based social music discovery and journaling platform designed to connect listeners through measurable similarities in their listening behaviour. At a surface level, users can:

- Create and edit reviews for songs, albums and playlists
- Build personal or collaborative playlists
- View personalised listening insights about what, when and how they listen
- Discover and connect with users who appear to have a similar taste profile

Behind these features, the project treats the system as an **experimental recommendation and analysis engine** rather than just a static website. Each user's listening history and interactions are converted into numerical feature vectors (e.g. artist frequencies, genre proportions, typical tempo and energy ranges). Similarity between users is then estimated using distance metrics such as **cosine similarity** and **nearest-neighbour style matching**, producing a "taste match" score that drives friend and content recommendations.

Playlists are analysed using a lightweight **mood classification**, which groups tracks based on combinations of features like energy, valence and tempo. Simple clustering and rule-based models will be compared to see how well they can label playlists as "chill", "focused", or "upbeat", and these labels are then surfaced in the UI as part of the user's music personality and discovery tools.

Overall, the project is framed as an experiment: it does not just present reviews and playlists but evaluates how far relatively simple recommendation and classification algorithms can support social connection, discovery and reflection within a music-focused web platform.

1.3 Project Aims and Objectives

This project aims to design, construct, and assess a web application with an experimental social aspect that integrates music journaling, personalized analytics, and community interaction. Fundamentally, it revolves around figuring out how various algorithms like similarity measures, mood classification, and ranking methods can be utilized to depict a user's music preference, find the most relevant content, and encourage the formation of music-based relationships that have a deepening effect.

Objectives

A) Technical Development

1. Create and launch the core OnChord platform as a full-stack web application (React, TypeScript, Node.js, Supabase) with a modular architecture that separates UI, application logic, data storage, and experimental machine learning components.
2. Specify and execute the data model and the processing pipeline for the depiction of users, reviews, playlists, social connections, and listening-related features in a manner that facilitates later analytics and recommendation tasks.
3. Build and connect a Python-based machine learning microservice (FastAPI) that can:
    - calculate taste similarity scores between users by utilizing the most suitable distance/similarity metrics
    - determine the mood of playlists by categorizing feature vectors
    - after that, perform an experimental evaluation of the quality of these algorithms in the app context

B) Design & UX

4. Using Figma, design a complete, accessible UI/UX design system that visually represents the interaction with the product through both light and dark-mode layouts, typography, and component patterns for reviewing, browsing, and social interaction via desktop and mobile.
5. Run the main user scenarios, writing and editing reviews, creating and managing playlists, collaborating in playlists, marking favourites, and viewing profiles and feeds in a way that these user journeys prominently reveal the underlying analytics and taste-matching features.

C) Research & Requirements

6. Carry out user research (surveys and/or interviews) to get an insight into how music users discover music, share opinions, and use stats, and then convert these insights into clear OnChord functional and non-functional product requirements.
7. Investigate current platforms and scholarly work (e.g. Letterboxd, Last.fm, and recommender-system literature) to figure out the most appropriate algorithms, similarity measures, and interface patterns, and then use this to explain the design decisions and the selection of ML techniques.

D) Testing & Evaluation

8. Evaluate system behaviour through structured testing, including unit and integration tests for core features, and targeted tests for data processing and taste-matching/mood-classification pipelines.
9. Evaluate usability, user experience, and algorithm performance by collecting feedback from test users and determining points where the ML outputs and recommendation logic perform well or poorly, thus recording limitations, technical challenges, and potential for later upgrades.

1.4 Project Scope

This project's scope sets the limits, anticipated features, and the constraints of the OnChord platform. It specifies the system's outputs that can be expected within the period of a final-year project and the ideas that won't be implemented. The emphasis is on four main areas: music journaling, social interaction, personalized insights, and experimental machine-learning features.

In-Scope Functionality

1. Core Platform Features
The project will implement the central features needed to support music-based journaling and discovery:

- Writing, publishing, editing, and deleting reviews for songs, albums, and playlists
- Creating and managing personal and playlists
- Creating collaborative playlists with selected users
- Viewing other users' public reviews, and favourites
- Following and unfollowing users, and browsing activity linked to those connections

These features form the minimum viable product (MVP) for OnChord.

2. Data and External Information Sources
To improve the experience, the system will use external music and event data. For example, by associating the real track/artist metadata to reviews and by displaying the next concerts related to the user's taste. Only a few endpoint selections will be used for track metadata, basic stats, and event listings; any large-scale data or analytics isn't considered.

3. Machine Learning Features (Experimental)
The project will include simple ML components through a Python FastAPI microservice:

- A **taste similarity algorithm**, using distance or similarity measures (e.g. cosine similarity) on user profiles out how close your music taste is to somebody else
- A **playlist mood classifier**, using simple supervised models (e.g. Random Forest or similar) to determine a created playlists mood

These models are treated as experimental: the goal is to make a prototype and evaluate their behaviour, not to deliver production-grade recommendations. Their limitations and failure cases will be documented.

4. User Interface & Experience
In scope:

- A complete frontend design in Figma
- A soft coloured, music-centric, responsive layout for desktop and mobile
- Navigation flow for core tasks (reviews, playlists, insights, profiles)

- Use of consistent components, modals and menus
- Basic accessibility considerations (colour contrast, legible typography, clear states)

The emphasis is on clarity of interaction rather than on advanced animations or visual effects.

5. Authentication & Settings
The system will provide:

- User registration and login
- OAuth login using a music account (e.g. Spotify)
- Basic profile and privacy settings (e.g. public/friends-only/private content)
- Simple appearance controls such as dark mode and accent colour selection

Out-of-Scope Functionality

To keep the project achievable, the following are explicitly excluded:

1. **Native Mobile Apps**
   No separate iOS/Android apps will be developed. The focus is on a mobile-responsive web application.
2. **Advanced or Large-Scale Machine Learning**
   Deep learning, large training datasets, real-time audio signal processing, and production-level recommendations are out of scope.
3. **Full Moderation and Reporting Tooling**
   Advanced moderation workflows such as reporting, automated abuse detection, or admin dashboards are not included. Basic privacy controls will exist, but community governance is beyond the scope.

Scope Conclusion

The scope is intended to make the proposal feasible. By focusing on essential social features, personalized insights, ML experiments, and a nicely designed interface, OnChord can show a considerable amount of technical depth while still being able to include a critical evaluation of which aspects function, which do not, and in which way the system could be taken further beyond the project.

1.5 Thesis Roadmap

This thesis is structured into seven chapters, each covering a key stage in the design, development, and evaluation of the OnChord platform.

Chapter 1 - Introduction
Introduces the project, explaining the motivation for OnChord, the problem space, the aims and objectives, and the overall scope of the system.

Chapter 2 - Literature Review
Reviews existing platforms, academic literature, final year projects, and relevant technologies

in social journaling, music analytics, and recommender systems to provide the foundation of the project.

Chapter 3 - System Analysis
Describes how requirements were gathered and refined, including stakeholder analysis, functional and non-functional requirements, use cases, and the initial system specification.

Chapter 4 - System Design
Outlines the architectural design of OnChord, covering the frontend, backend, data model, machine-learning components, and key UI/UX and system diagrams.

Chapter 5 - Testing and Evaluation
Presents the testing strategy and evaluation methods, including unit, integration, API, usability, and ML-related tests used to assess system quality.

Chapter 6 - System Prototype
Documents the implemented prototype, showing how the system will be integrated into actual code through Figma screenshots of the main features.

Chapter 7 - Issues and Future Work
Reflects on the limitations and challenges of the project and goes over future enhancements and directions for further development beyond the thesis.

2. Literature Review

2.1 Introduction

In this chapter there will be a review of existing platforms and technologies, academic literature, and past student projects that have guided the creation of OnChord. The review highlights weaknesses in music-based systems, examines suitable technologies, and studies scholarly works that underpin the project's viability and the selection of design.

2.2 Alternative Existing Solutions

OnChord sits at the intersection of **social journaling**, **music analytics**, and **music discovery**, drawing inspiration from several established platforms.

Social Journaling Platforms

Letterboxd:

A film journaling and review platform that allows users to log, list, and socially engage with movies.

Letterboxd is highly relevant because it proves the success of combining reviews, and social features in a single media-centric ecosystem. According to Forbes, its success is driven by strong community identity and expressive freedom.

Goodreads:

A social cataloguing platform for book reviews.

Goodreads demonstrates the value of reflective journaling and review-based discovery across another media domain.

Music Analytics Platforms

Last.fm:

Offers scrobbling and long-term listening analytics but lacks rich review features or social journaling.

Stats.fm (formerly Spotistats):
Provides Spotify-based analytics and insights but offers no meaningful social layer (comments, reviews, community).

Obscurify:

Generates personality-style listening statistics but offers no journaling or social features.

Gap identified:
All three platforms provide data, but none support expressive, community-driven interaction like OnChord.

Music Databases & Discovery Platforms

RateYourMusic (RYM)
A community-driven music database and rating site. Offers reviews but features a dated UI and lacks modern social mechanics.

Discogs
A marketplace and catalogue of music releases. Strong metadata modelling but not designed for reviews or music journaling.

Industry Context

Verified industry articles highlight the continued growth and evolution of streaming:

- **Global streaming growth (+4.8%) and 752M paid subscribers**
  Music Business Worldwide
- **Streaming has reshaped listening behaviour**
  World Economic Forum
- **Algorithmic ecosystems lack transparency; rise of AI issues**
  The Guardian

These reports support the need for platforms that support **social expression**, **reflection**, and **community**, not just passive algorithmic discovery.

2.3 Technologies Researched

OnChord is built using a modern web stack that balances familiarity (for faster development) with enough structure to support maintainability, modularity, and experimentation. This section explains why each technology was chosen rather than just listing tools.

Frontend Technologies

React - widely used in industry, has strong community support, and integrates well with modern tooling like TypeScript. Using React also aligns with material covered in "Internet Applications and Web Development".

TypeScript - used instead of plain JavaScript to add static typing to the frontend. Static types reduce common runtime errors and make it easier to safely change complex components such as the insight dashboard or review editor. TypeScript also keeps the codebase understandable and maintainable over the duration of the project.

Tailwind CSS - allows fast styling directly in JSX without maintaining large separate CSS files. This is particularly useful for a design-heavy app like OnChord, where many variants of cards, and layouts are needed.

Backend & Database

Node.js & Express - provides a lightweight, flexible environment that works naturally with JSON and REST APIs. It is well suited for building the kind of API endpoints OnChord needs which is authentication, review management, playlists, social interactions, and communication with the ML microservice.

Supabase - provides a managed PostgreSQL database, built-in authentication, and convenient developer tooling. PostgreSQL supports the relational data model needed for users, reviews, playlists, favourites, follows, and reminders, while still allowing JSON fields where flexibility is needed. Earlier in the project MongoDB Atlas was considered, but PostgreSQL was preferred because the data is naturally relational rather than document based.

Machine Learning Stack

Python - used for the machine learning microservice because it is the standard language for ML and data analysis. It offers a rich ecosystem of libraries and tutorials, which is important given the experimental nature of the ML features in OnChord and the limited project timeframe. It is also what I learned during my few weeks in my "Machine Learning" course

FastAPI - it's a framework for exposing ML models via HTTP endpoints. It is lightweight, easy to set up, and integrates well with Python's type hints.

scikit-learn, pandas, NumPy

- **NumPy** provides efficient numerical operations and array handling, which are needed when working with feature vectors (e.g. user taste profiles or playlist feature averages).
- **pandas** are used for loading, cleaning, and transforming tabular data for experiments or offline evaluation.
- **scikit-learn** offers a simple, well-documented implementation of common algorithms such as nearest neighbours, clustering, and simple classifiers (e.g. Random Forest), which are suitable for building the playlist mood classifier and similarity models without requiring deep learning or large-scale infrastructure.

These tools were selected because they are stable, well documented, and more than sufficient for the scale and experimental depth aimed for in this final-year project.

External APIs

Spotify Web API

https://developer.spotify.com/documentation/web-api

Provides:

- audio features (valence, energy, tempo)
- listening history
- top artists/tracks
- metadata

Spotify Tracks Dataset (Kaggle - "Ultimate Spotify Tracks DB")

https://www.kaggle.com/datasets/zaheenhamidani/ultimate-spotify-tracks-db

- audio features (valence, energy, tempo, danceability, etc)
- track-level metadata (name, album, artist, popularity)
- artist & genre info

Ticketmaster Discovery API

https://developer.ticketmaster.com/products-and-docs/apis/discovery-api/v2/

Provides:

- concerts
- events
- artist performances

2.4 Other Research

Recommender Systems

Academic works justify the ML components of OnChord:

- "Recommender Systems: The Textbook" - Aggarwal (2016) https://www.springer.com/gp/book/9783319296579
- "Recommender Systems Handbook" - Ricci, Rokach & Shapira (2022) https://link.springer.com/book/10.1007/978-1-0716-2197-4

These texts describe collaborative filtering, cosine similarity, hybrid models, and evaluation methods such as RMSE and precision@k.

Recommender systems in music

Some breakdowns of Spotify's recommendation engine explain that it mixes **collaborative filtering** (people who listen to similar music) with **content-based methods** (audio features and metadata) to personalise what users see. Music Tomorrow+2TechAhead+2 These sources support OnChord's decision to use simpler versions of the same ideas: represent each user as a taste profile and then compare those profiles using similarity scores.

Audio features and mood

Open papers and projects show that Spotify audio features like **valence**, **energy**, and **danceability** are meaningfully related to how people experience mood and intensity in music. Sciety+2PLOS+2 GitHub projects also demonstrate small-scale mood classifiers that use these features with classic ML models (e.g. logistic regression, KNN, ensemble methods) to label songs or playlists into simple categories like "happy", "sad", or "energetic". GitHub+3GitHub+3GitHub+3 This directly supports OnChord's idea of an experimental playlist mood classifier.

Social music discovery

Spotify's own research and case studies highlight that **sharing tracks with friends** plays a big role in actual music discovery and engagement, not just algorithmic playlists. AAAI

Digital Library+3Spotify Research+3arXiv+3 This backs up OnChord's focus on following users, viewing each other's reviews, and using taste matching to help people find others they might click with musically.

Overall, this research suggests that using audio features for mood, basic similarity measures for taste, and social features for discovery is a sensible and realistic direction for a final-year project.

2.5 Existing Final Year Projects

Two previous TU Dublin final-year projects were particularly useful in judging what is realistic for OnChord and what level of ambition is allowed.

Social Network Platform with Identity Verification
This project built a secure social media platform with full user verification using an external KYC service. It showed that it is possible, at FYP level, to design and implement a social system with user accounts, profiles, feeds and third-party API integration. For OnChord, this reassures that building a social layer (profiles, follows, visibility settings) on top of a web app is achievable within the timeframe, if the scope is managed carefully.

VR Music Learner
This project created a VR-based music learning environment with real-time feedback, audio analysis and a companion web/mobile interface. It demonstrated that music-related projects can successfully mix interactivity, audio features and educational or exploratory elements. OnChord might not use VR, but this project supports the idea that combining music, user interaction, and some level of intelligent processing is realistic at a FYP scale.

2.6 Conclusions

The research and examples reviewed in this chapter show that:

- Music listening today is heavily shaped by streaming and data-driven personalisation.
- There is still a gap for platforms that let people **talk about** music.
- Journaling systems like Letterboxd and Goodreads work well in other media and can be adapted to music.
- Simple, well-chosen algorithms can be used to model taste and mood in a realistic way at FYP scale.
- Social features, music analytics, and light ML can sensibly coexist in a single experimental web app.

Overall, the literature and existing systems support OnChord's core idea, which is a social, journaling-style music platform with personalised insights and experimental recommendation features. This forms the foundation for the more detailed system analysis and system design.

3. System Analysis

3.1 System Overview

OnChord is a web-based social platform where people can **journal about their music**, discover new artists, and connect with others who share similar tastes. Instead of just streaming songs, users can write reviews for albums and tracks, create playlists, find out the general mood or style of a playlist, and explore how similar their music taste is with other people.

The focus is on being **simple, friendly, and expressive**. OnChord is less about heavy data and more about giving users a space to talk about music, reflect on what they listen to, and see personalised insights that help them understand their taste.

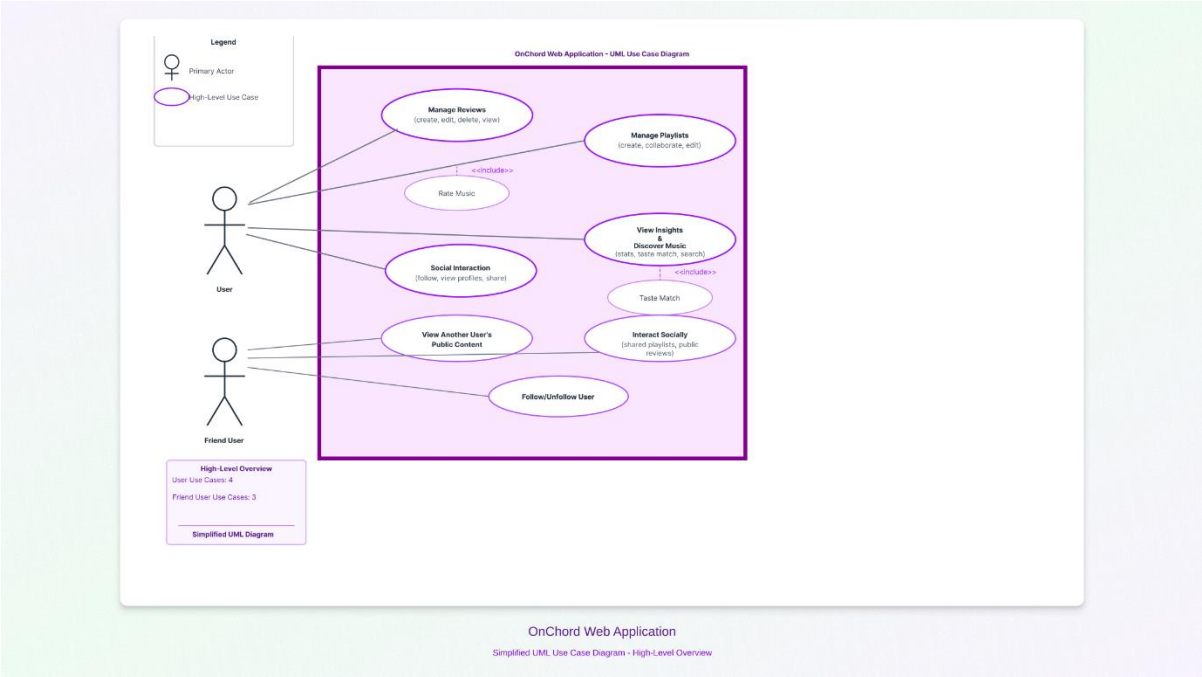
A typical user journey might look like this:

- Log in and sync their recent listening history
- Scroll the homepage to read friends' reviews and see what they've been playing
- Write a short review for an album or song they've just listened to
- Create a playlist or list, then check what overall mood or style it gets labelled as
- View a "taste match score to see how similar their music taste is to a friend's
- Visit another user's profile to browse their public reviews, and favourites for new music ideas

Everything takes place within one single, merged, dark, themed interface that is meant to resemble a Letterboxd, style experience for music, a place to log, share, and discuss what you listen to.

3.2 Requirements Gathering

<u>Use Case Diagram</u>



USE CASE	UC-01	**Manage Reviews (Create, Edit, Delete, View)**
Description of Goal in Context	The user wants to create, edit, delete, or view a review for a song, album, or playlist within the OnChord platform. This provides a journaling space for sharing their musical opinions with others.	
Preconditions	• User is logged in. • The chosen song/album/playlist is available in the interface.	
Post Conditions, Success End Condition	• A new review is created, or an existing review is updated or removed. • The user can see the correct review list on the relevant page (profile, album page, etc.).	
DESCRIPTION of Scenario	The user selects a music item and chooses to write or edit a review. The system shows a review editor where the user can enter text, rating, and visibility. After submitting, the review appears in the appropriate lists and views based on its visibility settings.	

Main Flow		
Step	**Action**	**Alternate**
n.1	User selects song/album/playlist	
n.2	System opens review editor/modal	
n.3	User enters text, rating, tags, visibility	
n.4	User clicks Publish or Save Draft	

n.5	System checks that required fields are filled in correctly	
n.6	System saves the review	
n.7	System refreshes the page to show the new or updated review	
EXCEPTIONS or ERROR Flow **Description** **EF1 -** Database Write Failure		
Step	**Branching Action**	**Alternate**
n.m.1	System cannot save the review	
n.m.2	System shows an error message ("Unable to save review")	
n.m.3	User can try again or close the editor	
ALTERNATIVE or VARIATION Flow **Description** AF1 - Editing Review		
Step	**Branching Action**	**Alternate**
n.m.1	User selects "Edit Review"	
n.m.2	System loads the existing review into the editor	
n.m.3	User changes the content and submits	
n.m.4	System replaces the old review with the updated version	

USE CASE	UC-02	**Manage Playlists (Create, Edit, Collaborate)**
Description of Goal in Context	The user creates, edits, or manages personal or collaborative playlists, adding or removing items and adjusting visibility.	
Preconditions	• User is logged in. • Music items exist in the system.	
Post Conditions, Success End Condition	• Playlist/list is created, modified, or deleted. • Any collaborative changes are reflected for all participants.	
DESCRIPTION of Scenario	The user opens the playlists section, creates a new playlist or list, adds music items, sets visibility, and optionally invites collaborators. The system updates the displayed accordingly.	

Main Flow

Step	**Action**	**Alternate**
n.1	User selects “Create List/Playlist”	
n.2	System shows creation form	
n.3	User adds name, description, visibility	
n.4	User searches music and adds items	
n.5	User clicks **Save**	
n.6	System saves the playlist/list	
n.7	UI updates with new list	

EXCEPTIONS or ERROR Flow

Description

EF1 - Playlist Save Failure

Step	**Branching Action**	**Alternate**
n.m.1	Save request fails	
n.m.2	Error message displayed	
n.m.3	User retries	

ALTERNATIVE or VARIATION Flow

Description

AF1 - Collaborative Playlist Creation

Step	**Branching Action**	**Alternate**
n.m.1	User selects “Collaborative Playlist”	
n.m.2	Adds collaborators	

n.m.3	System stores collaborator list	

USE CASE	UC-03	**Social Interaction (Follow, Share, View Profiles)**
Description of Goal in Context	The user performs social actions such as following users, viewing profiles, and sharing content, enabling community engagement.	
Preconditions	• User logged in. • Target user exists.	
Post Conditions, Success End Condition	• Follow relationships updated. • Shared content appears in feed.	
DESCRIPTION of Scenario	User opens another user's profile and chooses an action like follow or share. The system updates the social connection tables.	

Main Flow

Step	**Action**	**Alternate**
n.1	User opens another profile	
n.2	System shows profile details and follow status.	
n.3	User selects social action (follow/share)	
n.4	System updates the social state	
n.5	System refreshes the view to show the new status	

EXCEPTIONS or ERROR Flow

Description

EF1 - Social Action Fails

Step	**Branching Action**	**Alternate**
n.m.1	System cannot apply the requested action	
n.m.2	System displays an error message	

ALTERNATIVE or VARIATION Flow

Description

AF1 - User Shares Content

Step	**Branching Action**	**Alternate**
n.m.1	User chooses "Share"	
n.m.2	System opens share modal	

n.m.3	User selects where to share	
n.m.4	System posts shared content	

Stakeholders

1. **Primary Users** - everyday music listeners who want to review, document, and share their music experiences.
2. **Social Users** - individuals interested in connecting with others based on shared musical interests.
3. **Creators / Curators** - users who enjoy building playlists, or recommendations for others.
4. **System Administrators** - responsible for maintenance, performance monitoring, and security.

Requirements Collection Methods

To gather initial requirements, several research and data collection methods were used:

1. User Survey (Google Forms)

A structured questionnaire was created to understand user expectations around:

- Music reviewing
- Playlist creation
- Social features
- Analytics and statistics
- Machine learning insights

The survey results showed strong interest in:

- Review visibility options
- Personalised music insights
- Collaborative playlists
- Social connections based on taste similarity
- A clean and visually appealing UI

This validated the inclusion of features such as taste match, reviews, and favourites.

2. Competitor Analysis

Platforms like Letterboxd, Stats.fm, Last.fm, and RateYourMusic were analysed to identify:

- Strength's worth adopting
- Limitations to avoid

- Missing features that OnChord can fill

This analysis guided features like:

- Review journaling
- Music insights
- Social features

3. Figma Design Analysis

By developing early UI concepts, it was possible to identify:

- Navigation complexity
- Required pages
- Modal interactions
- Data flow requirements

4. Academic and Industry Research

Studies on recommender systems, UX design, and social media behaviour provided additional guidance on:

- ML feasibility
- Best practices for dark-mode interfaces
- Social engagement patterns in digital media platforms

Initial Requirements Identified

From these methods, the following high-level requirements were gathered:

- The system must allow users to write, edit, and manage music reviews.
- Users must be able to collaborative playlists with others.
- The system should generate personalised insights using algorithms such as matching with other people's music taste, and basic playlist mood classification.
- Recommendations should be based on users' listening behaviour and past interactions (reviews, favourites, follows).
- The interface must be accessible, responsive, and visually consistent.
- Authentication must be secure and support modern sign-in flows.
- Users must be able to follow or unfollow others and see relevant updates.
- Favourites and lists must be clearly structured so users can quickly find and organise music.

These requirements form the foundation for detailed modelling and specification.

3.3 Requirements Analysis

The purpose of the requirements analysis is to turn the functional and non-functional requirements into a clear initial systems model, showing how the different elements of

OnChord interact and how the system should behave. This model forms the bridge between requirement gathering (what the system must do) and system design (how it will be built).

The initial model was developed using insights from the Use Case Diagram, survey findings, functional requirements, non-functional requirements, and technical feasibility considerations. These collectively define the major components of the system and the actors involved.

3.3.1 Actors and System Boundaries

From the requirements, the primary actors identified are:

- **Main User -** interacts with all core platform features
- **Friend User -** views public content, interacts socially, follows/unfollows

This establishes a clear boundary: the user only interacts with the UI, while all data logic, API communication, authentication flows, and ML processing occur within the system layers.

3.3.2 Core System Processes

Analysis of the requirements reveals that OnChord's behaviour can be grouped into the following core processes:

1. **Authentication & Account Management**
    - Login
    - Account settings
2. **Content Creation & Management**
    - Writing, editing, and deleting reviews
    - Creating lists and collaborative playlists
    - Adding favourites
3. **Social Interaction**
    - Following/unfollowing
    - Liking, commenting
4. **Music Insights & Analytics**
    - Retrieving listening data
    - Taste similarity
    - Playlist mood classification
5. **Search & Discovery**
    - Searching music and users
    - Viewing metadata and previews
6. **Events & Reminders**

- Concerts
- Calendars
- Notifications

These processes reflect the functional requirements and directly map to the use cases defined earlier.

3.3.3 Data Entities and Relationships

Based on the functional requirements and survey-driven needs, the following core data entities were identified:

- **User** (profiles, settings, preferences)
- **Reviews** (rating, text, visibility)
- **Collaborative Playlists**
- **Favourites** (songs, albums, artists)
- **Social Graph** (followers, following)
- **Events & Reminders** (calendar items)

Analysis of these entities informs the system's data model, showing clear relationships such as:

- A **User** can have many **Reviews**, **Playlists**, and **Favourites**
- A **Playlist** can belong to multiple **Users** (collaboration)
- A **User** can follow many other **Users** (social graph)
- A **Reminder** must be associated with a **User**

This reflects the conceptual ERD that will be expanded in Chapter 4.

3.3.4 System Architecture Model (Conceptual)

From analysing requirements, the system will adopt a **three-tier modular structure**:

1. Presentation Layer (Frontend)

- React + TypeScript UI
- Displays reviews, playlists, profiles, insights
- Handles user interactions and inputs

2. Application Layer (Backend API)

- Node.js + Express
- Processes requests, validates data, manages sessions
- Communicates with APIs

- Routes data to/from database and ML microservice

3. Data/ML Layer

- **Database (Supabase)** stores all user-generated content
- **ML Microservice (FastAPI + Python)** performs:
  - Taste similarity
  - Playlist mood classification

This layered separation is consistent with the system's functional requirements and helps maintain scalability and maintainability.

3.3.5 Mapping Requirements to the System Model

Each functional and non-functional requirement aligns with one or more system components. This mapping ensures that every feature defined in Section 3.X is achievable within the architecture and that the system behaves predictably across all modules. The table below shows how each requirement category integrates with the technical layers identified in the initial systems model.

Requirement-to-Component Mapping

Requirement Area	System Component
Reviews, playlists, social	Frontend + Backend + Database
OAuth login	Backend
Insights & ML	Backend + ML Microservice
Events	Backend
Search	Backend + Frontend
Profiles	Backend + Database

This mapping highlights how individual requirement groups depend on multiple parts of the architecture. For example:

- **Reviews, playlists, and social features** require coordination between the UI (Frontend), business logic (Backend), and persistent storage (Database).
- **Insights and machine learning** rely on data flow to the Backend, then onward to the ML Microservice.
- **Search** involves UI interactions (query input, filter selection) and Backend processes for database querying and result retrieval.
- **Profiles** are read-heavy operations dependent on backend data aggregation and fast database retrieval.

3.3.6 Initial System Model Summary

The initial systems model represents a conceptual blueprint for how OnChord will operate. It synthesises user requirements, stakeholder needs, technical constraints, and platform goals into a structured view of the system's behaviour and architecture. This model lays the groundwork for the detailed system design presented in Chapter 4.

Actors

The model identifies several actors that interact with the system:

- **User:** Creates content (reviews, playlists, favourites), explores insights, manages personal data, interacts socially.
- **Friend User:** Views other users' public content, follows or unfollows users, likes or comments on reviews.

Each actor clearly defines a set of interactions, helping establish user-system boundaries and internal responsibilities.

Processes

The main system processes derived from the requirements are:

- **Reviewing and Journaling:** Creating, editing, deleting, and displaying reviews.
- **Playlist Management:** Building collaborative playlists, adding or removing items.
- **Social Interaction:** Following users, liking reviews, posting comments.
- **Personalised Insights:** Analysing music taste to other uses and playlist moods through ML predictions.
- **Event Interaction:** Viewing concerts, adding events to calendars.
- **Search & Discovery:** Searching music, users and playlists across the platform.

These processes directly correspond to the functional requirements and Use Case Narratives, ensuring alignment between user expectations and system behaviour.

Data Entities and Relationships

Through analysis, the following core data entities were identified:

- **User**
- **Review**
- **Playlist**
- **Collaborative Playlist**
- **Favourite Item**
- **Social Graph**
- **Reminder**
- **Event**

Key relationships include:

- A **User** can have many **Reviews**, **Playlists**, and **Favourites**.
- A **Playlist** may have multiple collaborating **Users**, forming a many-to-many relationship.
- A **User** can follow multiple other **Users**.
- A **Reminder** is always linked to exactly one **User**.
- **Events** retrieved from Ticketmaster are not stored permanently but may be linked to Reminders.

Architectural Layout

The initial systems model proposes a **four-component modular architecture**:

1. Frontend (React + TypeScript)

2. Backend API (Node.js + Express)

3. Database (Supabase / PostgreSQL)

4. ML Microservice (FastAPI + Python)

The separation of concerns enhances system maintainability, scalability, and clarity of responsibility.

System Boundaries

The initial systems model clarifies the distinct boundaries between:

User-Facing Operations

- UI interactions
- Reviewing content, browsing profiles, editing playlists
- Viewing insights and interacting socially

Internal System Logic

- Authentication handling
- Request validation
- Data transformation and aggregation
- Routing between services

External Services

- API calls
- Machine learning inference via FastAPI
- OAuth token management

This separation ensures secure design practices, efficient resource allocation, and easier debugging and scaling.

3.4 Initial System Specification

This section documents the initial functional and non-functional requirements of the OnChord system and outlines the most appropriate architecture to support these requirements.

3.4.1 Functional Requirements (FRs)

Functional requirements describe the specific behaviours and capabilities that the system must provide. These requirements were derived from the use case narratives, user survey results, platform comparisons, and the core goals of OnChord.

A. Authentication & User Accounts

- **FR1.** The system shall authenticate users using Spotify OAuth.
- **FR2.** The system shall allow users to update profile details and account settings.

B. Music Reviews & Journaling

- **FR3.** Users shall be able to write, edit, delete, and publish music reviews.
- **FR4.** Users shall be able to set review visibility (public, private).
- **FR5.** Reviews shall be displayed in user profiles and feeds.

C. Playlists & Favourites

- **FR6.** Users shall be able to create and manage collaborative playlists.
- **FR7.** The system shall allow users to favourite songs/albums.

D. Social Interaction

- **FR8.** The system shall allow users to follow and unfollow others.
- **FR9.** Users shall be able to like, comment on, and share reviews.
- **FR10.** The system shall show relevant social interactions like follows or likes.

E. Search & Discovery

- **FR11.** Users shall be able to search for music, users, playlists.

F. Insights, Analytics

- **FR12.** The system shall retrieve listening history and audio features.
- **FR13.** The system shall return taste similarity scores.
- **FR14.** The system shall classify playlist moods.

G. Events, Calendar & Reminders

- **FR15.** The system shall display concerts from Ticketmaster.
- **FR16.** Users shall be able to add events to their calendar.
- **FR17.** The system shall allow reminders to be created, modified, and viewed.

H. System Operations

- **FR18.** The system shall validate and sanitise all user input.

- **FR19.** The system shall support mobile-responsive functionality.

3.4.2 Non-Functional Requirements (NFRs)

Non-functional requirements define the performance, security, usability, scalability, and reliability expectations of the system. These requirements ensure high-quality operation.

A. Performance

- **NFR1.** API calls should complete within 3 seconds during standard operation.
- **NFR2.** ML predictions should return within 2 seconds.
- **NFR3.** Page loads should not exceed 2.5 seconds.

B. Security

- **NFR4.** All authentications must comply with OAuth 2.0 and Spotify API policies.

C. Usability

- **NFR5.** The UI must follow HCI guidelines.
- **NFR6.** Key user actions must require no more than three clicks from the homepage.
- **NFR7.** The interface must support dark mode.

D. Reliability

- **NFR8.** The system must maintain 95% uptime during deployment.
- **NFR9.** The system must remain functional during external API outages.
- **NFR10.** Daily database backups must be maintained.

E. Scalability

- **NFR11.** The database must support increasing volumes of reviews, playlists, and user interactions.
- **NFR12.** The architecture must allow components to scale independently.

F. Maintainability

- **NFR13.** The codebase must use well-documented components.
- **NFR14.** Version control must be enforced for all services (frontend, backend, ML).

3.4.3 Appropriate System Architecture

Based on the functional and non-functional requirements, the most appropriate architecture for OnChord is a **service-oriented, three-tier design** with a lightweight microservice for machine learning.

1. Frontend Layer - React + TypeScript

Responsible for:

- UI rendering
- Client-side routing

- Review creation, playlist management
- Displaying insights, events, and social information
- Communication with backend through REST API calls

The use of React supports reusable components, state management, and responsive layouts.

2. Backend Layer - Node.js + Express

Acts as the central orchestrator:

- Handles authentication (Spotify OAuth)
- Validates data and manages sessions
- Routes requests to the database, ML service, and external APIs
- Enforces permissions and visibility rules

The backend ensures security and logical control flow.

3. Data Layer - Supabase (PostgreSQL)

Stores all persistent platform data:

- Users
- Reviews
- Playlists/collaborative playlists
- Favourites
- Social graph
- Reminders

Supabase handles real-time updates, authentication metadata, and efficient relational queries.

4. ML Microservice - FastAPI + Python

Operates independently to allow scalable ML functionality:

- Taste similarity calculations
- Playlist mood classification
- Potential future ML experiments

A microservice architecture isolates ML processing, ensuring the main application stays responsive even under heavy analytical loads.

5. External Services

- **Spotify API -** Audio features, user listening data, track metadata
- **Ticketmaster API -** Concerts and events

External APIs extend platform capabilities without storing unnecessary data.

Summary of Architecture Choice

This architecture was selected because it:

- Supports the functional requirements through clear separation of concerns
- Meets the non-functional needs for scalability, maintainability, and reliability
- Allows the ML service to evolve independently of the main application
- Follows industry best practices taught across modules including **Web Development, Databases, Cloud Computing, OOP, and HCI**
- Ensures the system can expand over time without major changes

3.5 Conclusions

Chapter 3 defined what OnChord needs to do and how it should behave from a user-centred point of view. Through surveys, competitor analysis, Figma prototypes and research, I identified the main stakeholders, clarified the core user journeys, and captured a clear set of functional and non-functional requirements. These were then expressed as use cases, key processes (reviews, playlists, social features, insights, events, search), and core data entities such as users, reviews, playlists, favourites, and social connections.

From this analysis, an initial systems model and architecture were outlined: a React frontend, Node.js/Express backend, Supabase/PostgreSQL data layer, and a separate FastAPI/Python ML service. Together, these give a realistic blueprint for how the platform will support journaling, social interaction, and experimental recommendation features, and they provide a solid foundation for the detailed design work in Chapter 4.

4. System Design

4.1 Introduction

The goal of this chapter is to elaborate and convert the system requirements that were outlined in Chapter 3 into the design that is detailed and organised. It is a phase that connects the gap between the system functionalities and the way it will be implemented. The methodology is a mixture of software engineering principles, OnChord architectural modelling, interface design, database design, and workflow diagrams which serve as a proof that the project's feasible and maintainable

The design journey is enriched with the requirements analysis. It comprises functional requirements, non-functional requirements, and use case modelling, all of which are utilized to draft a blueprint. The plan covers the evolvement of the logical and physical architectures, determination of the system components, specification of the data structures, user interface design, and the communication flow establishment amongst the modules (frontend, backend, database, and machine learning microservice) by means of which interaction is realized.

4.2 Software Methodology

An Agile-inspired iterative methodology is the chosen method to develop OnChord efficiently. Regardless of it being a solo project, the use of Agile practices allows for flexibility, quick feedback, and ongoing refinement, which are especially helpful in a case where a system comprises a frontend design and experimental machine learning components.

Key Justifications for Agile Approach

- **Iterative development:** Allows individual features (reviews, playlists, ML insight) to be built, tested, and refined in short cycles.
- **Continuous integration:** Using GitHub Actions supports early detection of errors and encourages stable deployments.
- **User feedback integration:** Early Figma prototypes and survey responses inform design decisions and can be iteratively incorporated.
- **Adaptability:** APIs, ML models, or database structures can be adjusted as technical challenges emerge.
- **Risk reduction:** The experimental ML features can be developed in parallel, isolated from the main system, as recommended in modern Agile pipelines.

Methodology Techniques Used

- Kanban-style task board (Backlog → In Progress → Testing → Done)
- Sprint-like weekly development cycles
- Regular refactoring and incremental improvement
- Continuous documentation updates

- Modular component development (inspired by OOP and Systems Architecture modules)

This methodology supports a scalable and maintainable workflow suitable for both the complexity and experimental nature of OnChord. It's also a technique that I picked up during my Semester 2 Internship with Global Innovation.

4.3 Overview of System

This section provides a full architectural overview of the OnChord platform. It describes how the system is structured, how its major components communicate, and how the platform will be deployed in a production environment. The goal is to demonstrate how the system transforms the requirements outlined in Chapter 3 into a coherent and well-defined design.

OnChord follows a multi-layered architecture combining:

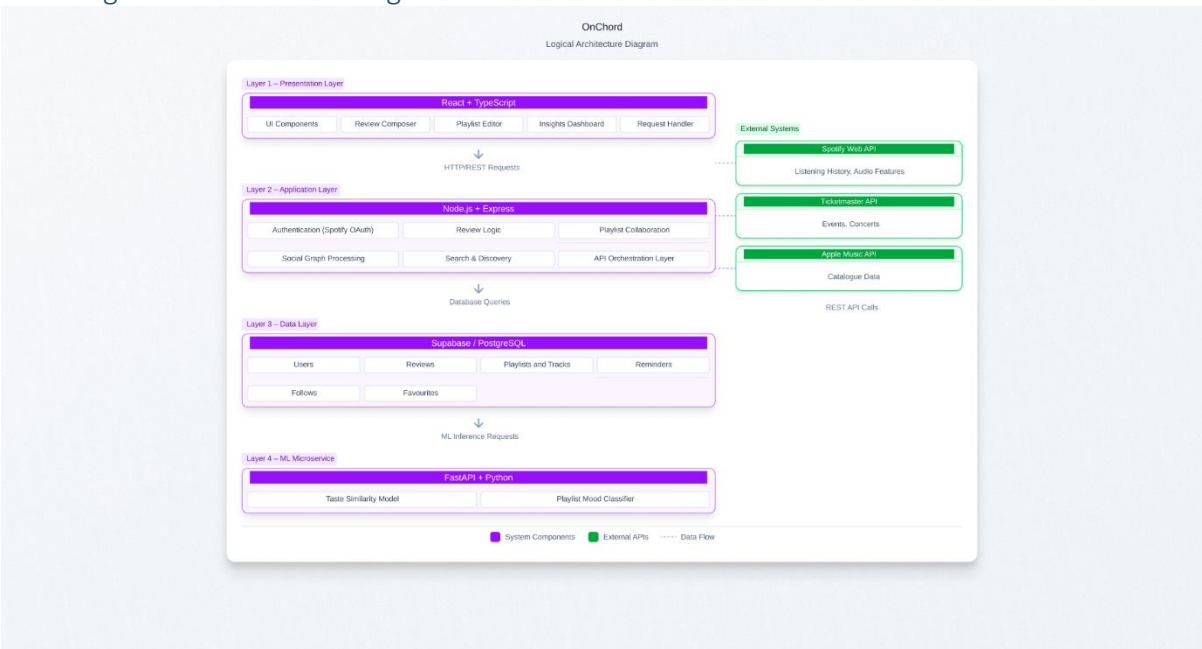
- A frontend presentation layer
- A backend application layer
- A cloud-hosted relational database
- A dedicated Python-based machine learning microservice
- Connections to multiple external APIs

The following subsections outline the system architecture using three key diagrams:

1. **Logical Architecture Diagram**
2. **Physical Infrastructure Diagram**
3. **Deployment Diagram**

Each diagram is accompanied by an explanation of its purpose and relevance to the system.

4.3.1 Logical Architecture Diagram



1. Presentation Layer (React + TypeScript)

This layer is responsible for everything the user sees and interacts with, including:

- UI components
- Review composition interface
- Playlist editor
- Insights dashboard
- Calendar and reminders interface
- Network request handling

The frontend communicates with the backend using **HTTP/REST requests** and receives JSON responses. The UI is designed using the prototypes created in Figma.

2. Application Layer (Node.js + Express)

This layer processes user actions, enforces business rules, and coordinates interactions across the platform. It includes the following services:

- **Authentication Service (Spotify OAuth)**
- **Review Service**
- **Playlist and Collaboration Service**
- **Insights & Statistics Service**
- **Social Graph Service (Follows, Interactions)**
- **Search & Discovery Service**
- **API Orchestration Layer** for handling external API communication

This layer also handles validation, caching, and performance optimisation.

3. Data Layer (Supabase / PostgreSQL)

This layer stores all persistent data that OnChord requires, including:

- User profiles
- Reviews
- Playlists
- Follows (social graph edges)
- Favourites
- Reminders
- Cached event metadata

Supabase provides PostgreSQL hosting, built-in authentication, row-level security, automated backups, and REST endpoints where required.

4. Machine Learning Layer (FastAPI + Python)

This layer provides experimental ML capabilities used for personalised insights. It includes:

- **Taste Similarity Model** - compares user listening patterns
- **Playlist Mood Classifier** - analyses a playlist based on audio features

The backend sends normalised input data to this microservice and receives prediction scores or classifications. Keeping the ML separate makes the system modular, maintainable, and easier to evaluate.

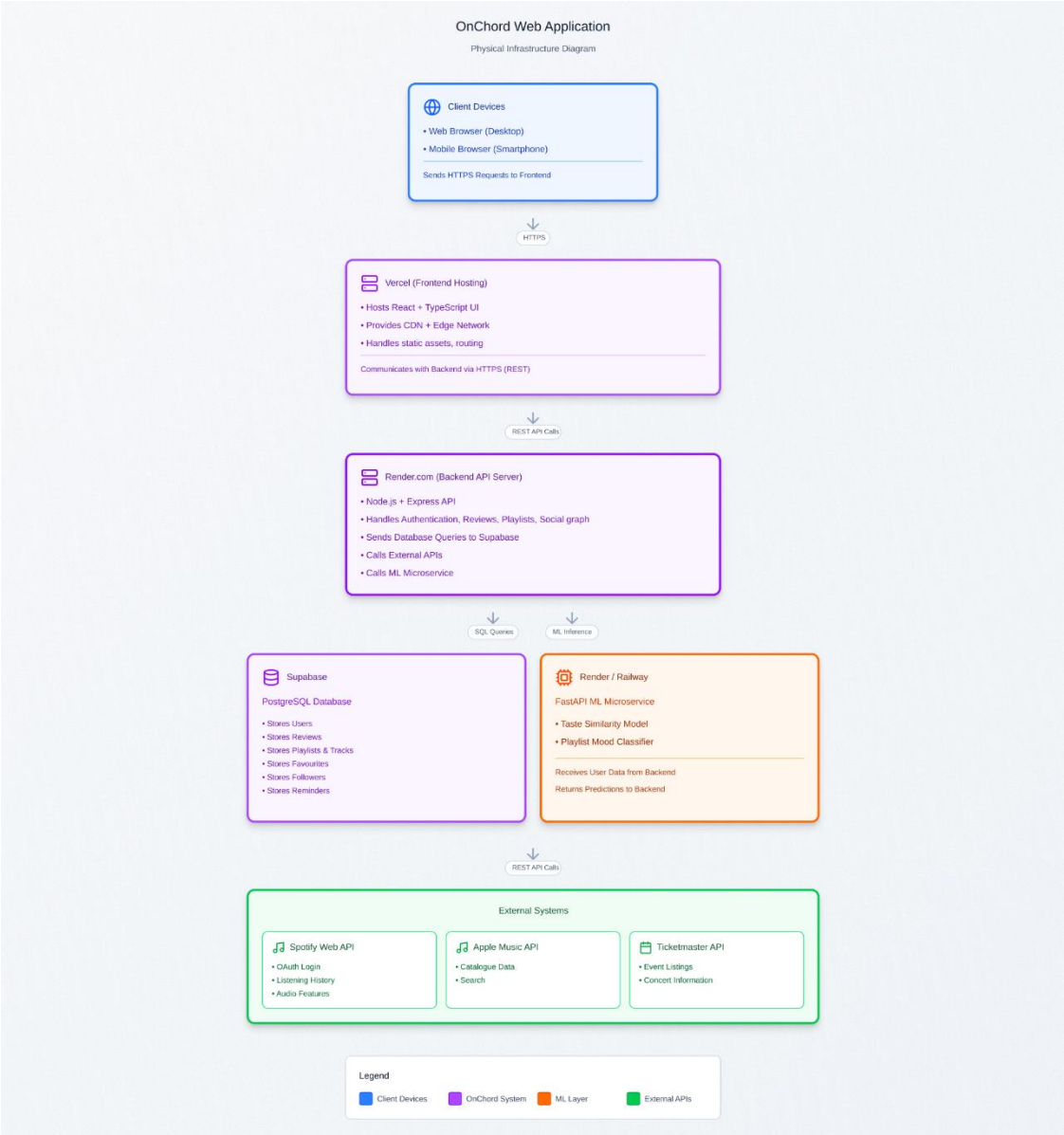
External Systems (APIs)

OnChord integrates with:

- **Spotify Web API** - listening history, audio features, track metadata
- **Ticketmaster API** - event and concert data
- **Apple Music API (possibly) - catalogue data**

These APIs enable OnChord to provide personalised recommendations, music metadata, and event reminders.

4.3.2 Physical Architecture Diagram



1. Frontend Hosting - Vercel

The React/TypeScript frontend is deployed on **Vercel**, which provides:

- Automatic builds from GitHub
- Serverless environment for static file hosting
- Fast edge caching for performance

Users access the frontend through their browser, which sends HTTPS requests.

2. Backend Hosting - Render.com (Node.js Server)

The backend API is hosted on **Render.com**. It handles:

- Authentication & OAuth
- Review, playlist, and favourite

- Social graph processing
- Search and discovery
- API orchestration for external services
- Communication with Supabase and the ML microservice

Render provides autoscaling, logging, environment variables, and reliability guarantees.

3. Database Hosting – Supabase (PostgreSQL)

Supabase's PostgreSQL instance provides:

- Relational data storage
- Authentication and user tables
- Real-time database subscriptions
- Automatic daily backups

The backend connects to Supabase via secure SQL queries.

4. ML Microservice Hosting – Render or Railway

The Python FastAPI ML service is hosted separately from the backend to ensure:

- Isolation of experimental features
- Independent scaling
- Lower risk of backend downtime
- Ability to replace models without affecting core system logic

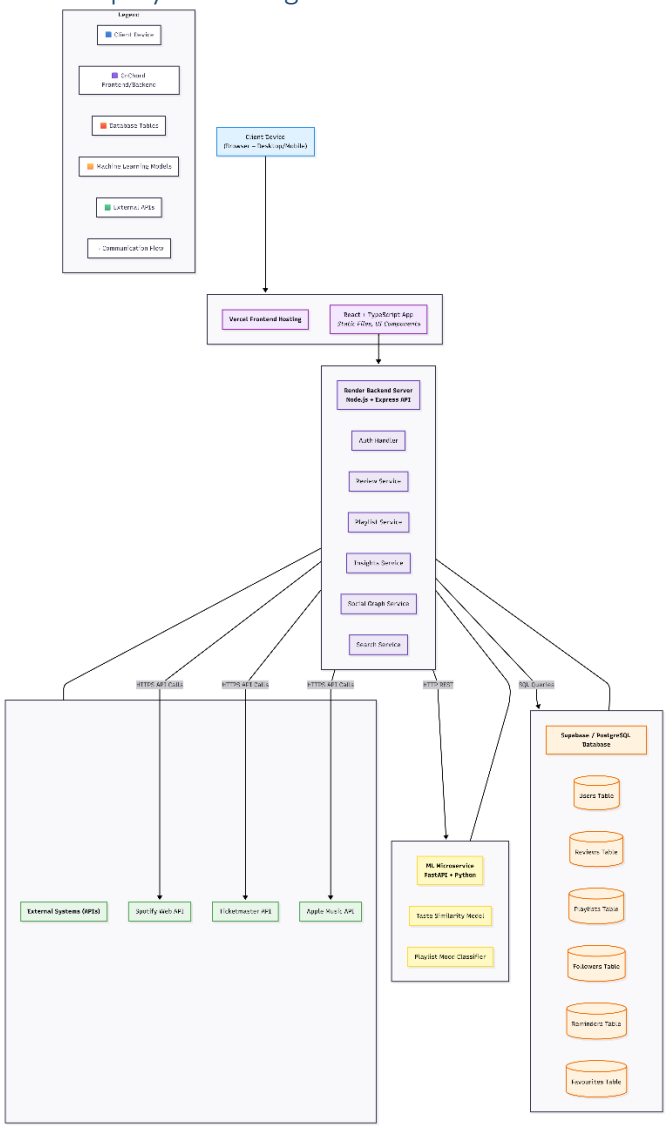
5. External APIs

These external systems provide live data:

- **Spotify** - track metadata, audio features
- **Ticketmaster** - concerts, events

All communication occurs over secure HTTPS.

4.3.3 Deployment Diagram



The deployment diagram visualises how the system behaves at runtime and how components exchange information:

Client Device → Frontend

Users access the application through:

- Desktop browsers
- Mobile browsers

Communication uses secure **HTTPS**.

Frontend → Backend

The frontend sends REST requests for:

- Logging in
- Creating/editing reviews
- Fetching insights

- Managing playlists
- Searching for users or music
- Managing favourites and reminders

Backend → Database

The backend performs SQL operations:

- Insert/update/delete operations for user-generated content
- Fetch operations for insights and analytics
- Query optimisation for feed generation and social interactions

Backend → ML Microservice

The backend sends inference requests when:

- Generating taste similarity scores
- Generating playlist mood classification

The microservice returns JSON responses.

Backend → External APIs

The backend handles all external API communication (the frontend never communicates with APIs directly):

- Spotify OAuth + track analysis
- Ticketmaster event lookups

4.4 Design System

4.4.1 User Interface & Experience Design (UI/UX)

The OnChord UI is designed using **Figma**, informed by UX principles from "Human Computer Interaction" module and by early user research through the Google Forms survey.

Design Goals

- **Clarity:** Clean navigation structure with consistent spacing, typography, and iconography.
- **Accessibility:** High contrast dark-mode interface (user preference from survey).
- **Consistency:** Component-based layout that mirrors React's modular structure.
- **Personalisation:** Dynamic content such as listening stats, collaborative playlists, and friend activity.
- **Minimal Cognitive Load:** Interfaces avoid clutter and highlight only relevant actions.

Core Screens Designed

- Authentication & onboarding
- Homepage feed
- Music review page
- Review editor
- Playlist creation and collaboration
- Taste Insights dashboard
- Calendar & reminders
- Settings & account pages
- User profiles (public vs private view)

Figma prototypes are also used to derive use cases and to define UI flows for implementation.

4.4.2 User Interface & Experience Design (UI/UX)

The system is decomposed into modular components aligned with React, Node.js, Supabase, and the FastAPI microservice.

Frontend Components (React)

- **Navbar Component**
- **ReviewComposer Component**
- **ReviewCard Component**
- **PlaylistEditor Component**
- **CalendarView Component**
- **InsightsChart Components (Radar, Bar, Timeline)**
- **UserProfile Component**
- **FollowButton Component**
- **SearchBar Component**

Each component encapsulates UI logic and communicates with the backend via a shared request handler.

Backend Components (Node + Express Services)

Each service matches a use case group identified in Chapter 3:

Service	Responsibilities
Auth Service	OAuth, JWT handling, session validation
Review Service	CRUD reviews, visibility rules
Playlist Service	Create/edit playlists, collaborative mode

Service	Responsibilities
Insights Service	Fetch user stats, interface with ML
Social Graph Service	Follow/unfollow, friend recommendations
Search Service	Database search + external catalog search
Notification Service (optional future work)	Reminders, email/Push notifications

4.4.3 User Interface & Experience Design (UI/UX)

This section describes how data moves through the system in a typical workflow.

Example Data Flow: Creating a Review

1. User writes a review on the frontend.
2. React sends POST /reviews to the backend.
3. Backend validates input, stores review in Supabase.
4. Backend returns success + updated review object.
5. Frontend updates UI and local state.

This layered flow ensures separation of concerns and maintains a secure, scalable architecture.

4.4.4 Data Flow & Interaction Design

The machine learning microservice is intentionally decoupled from the main backend to allow:

- independent updates to ML models
- experimentation without breaking core functionality
- scalable prediction handling

ML API Endpoints (FastAPI)

Endpoint	Purpose
/predict/taste_similarity	Returns similarity score between users
/predict/mood_classification	Classifies a playlist's mood using audio features

Model Workflow

1. Backend formats the input data (normalisation, feature extraction).
2. Backend sends JSON payload to the ML service.
3. ML model processes the data with scikit-learn.
4. Output returned as JSON with prediction + confidence score.

The ML integration is still experimental, but it aligns with FYP expectations of innovation.

4.4.5 Machine Learning Integration Design

External API Use

OnChord communicates with:

E) Spotify Web API

- OAuth
- Track metadata
- Listening history
- Audio features

F) Ticketmaster API

- Event metadata
- Concert listings
- Venue information

API Routing Structure

The backend includes an orchestration layer to:

- unify API responses
- manage rate limits
- handle caching
- protect API keys
- normalise data into a shared schema

This prevents the frontend from interacting directly with any external API, improving security and performance.

4.4.6 Database Design (Supabase / PostgreSQL)

This section connects back to 4.X where the ERD is fully explained and confirms how the database aligns with the system design.

The schema supports:

- 1:M review relationships
- collaborative playlists
- many-to-many follows
- reminders and event caching
- favourites for multiple item types

Foreign keys, indexing, and cascading rules were chosen to ensure consistency and performance.

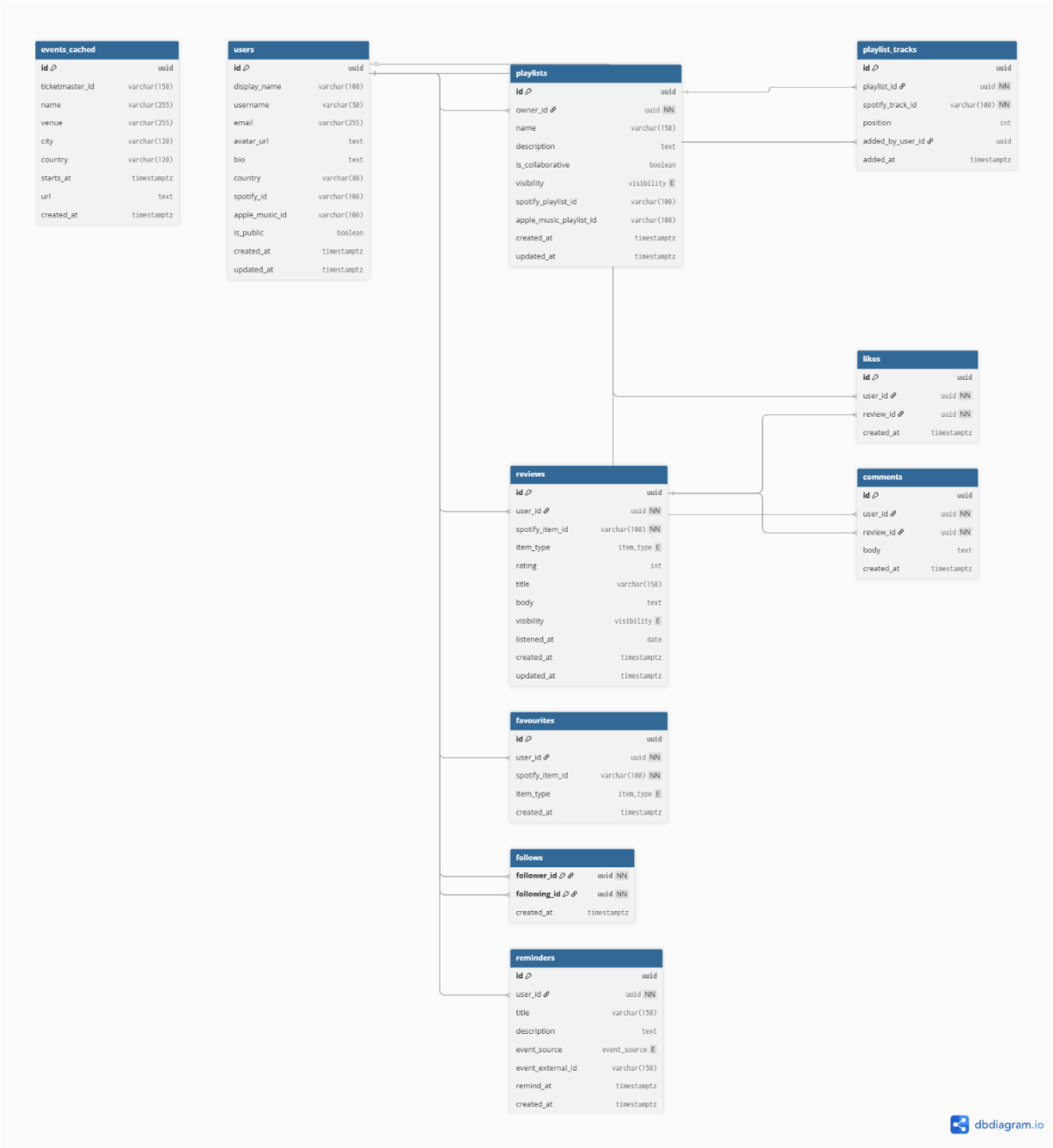
4.4.7 Summary

The system design defines how all components of OnChord interact and how the platform will be implemented in the next chapter. This includes:

- UI and user journey design
- Modular component breakdown
- Backend services and controllers
- Data flow processes
- Machine learning architecture
- API integration
- Database structures

These models collectively form the implementation blueprint and ensure that the system is technically sound, scalable, and aligned with the requirements defined earlier.

4.5 Database Design - ERD Explanation



The Entity-Relationship Diagram (ERD) shown above represents the core data structures required to support all functional elements of the OnChord platform. The database is implemented using PostgreSQL via Supabase, which provides relational integrity, strong consistency, and support for JSON-based metadata when needed.

The ERD captures all major entities identified during requirements analysis, including users, reviews, playlists, social relationships, reminders, and event caching. The design follows a normalised relational model to reduce redundancy, preserve data integrity, and optimise query performance.

Key Entities and Their Roles

1. Users

The **users** table stores core account information such as username, email, display name, Spotify identifiers, and privacy preferences.
It forms the foundation of almost every relationship in the system.

A user can be associated with:

- multiple reviews
- multiple playlists or lists
- favourites
- reminders
- the social follow graph

This aligns with the requirement for personalised content and social interactions.

2. Reviews

The **reviews** table stores user-written reviews of songs, albums, or playlists.
Key attributes include:

- rating
- review text
- visibility settings
- type of media (track/album/playlist)
- timestamps

Each review is linked to its author through user_id, ensuring a strong 1:M relationship between users and reviews.

3. Playlists & Playlist Tracks

This subsystem contains two linked tables:

- **playlists**

Stores playlist metadata, including:

- owner
- whether the playlist is collaborative
- visibility
- optional references to Spotify playlist IDs
- **playlist_tracks**

Stores individual tracks within a playlist.
Includes:

- the track's Spotify ID
- order/position
- the user who added the track

This design supports collaborative playlist workflows by allowing multiple users to modify a single playlist entity.

5. Favourites

The **favourites** table allows users to bookmark songs, albums, or playlists.
This supports "Favourite Songs" and "Favourite Albums" pages inside the application.

6. Social Graph (Follows)

The **follows** table models user relationships using a self-referential structure:

follower_id → users.id

following_id → users.id

This enables:

- social feeds
- user discovery
- taste matching between connected users

The primary key is a composite pair to ensure no duplicates.

7. Likes & Comments

These tables model interactions on reviews:

- **likes** allow users to like specific reviews
- **comments** allow threaded discussion under reviews

Each comment and like references both the review and the user involved.

8. Reminders & Events

The **reminders** table supports:

- calendar events
- concert reminders
- custom reminders

Events fetched from Ticketmaster are cached in **events_cached**, improving performance and reducing API calls.

Relationship Summary

- **Users ⟶ Reviews** : 1-to-Many
- **Users ⟶ Playlists** : 1-to-Many
- **Playlists ⟶ Playlist Tracks** : 1-to-Many
- **Users ⟶ Favourites** : 1-to-Many

- **Users ⟶ Reminders** : 1-to-Many
- **Users ⟶ Users (Follows)** : Many-to-Many through a self-referencing table
- **Reviews ⟶ Likes** : 1-to-Many
- **Reviews ⟶ Comments** : 1-to-Many

This relational structure ensures OnChord can efficiently handle:

- high volumes of user-generated content
- scalable social interactions
- personalised analytics
- rapid retrieval of music-related data

4.6 Conclusions

This chapter changed the necessary things in Chapter 3 to a real, workable design for OnChord. The general structure has been outlined as a multi-layer system with a React/TypeScript frontend, a Node.js/Express backend, a Supabase/PostgreSQL data layer, and a separate FastAPI/Python microservice for machine-learning features such as taste similarity and playlist mood classification Logical, physical, and deployment diagrams illustrated how these elements interacted, how data flowed between them, and how the system would operate in a real cloud environment.

This chapter also defined the UI/UX direction via Figma prototypes, divided the platform into modular frontend and backend services, explained data-flow patterns for typical operations like creating reviews, and coordinated the database schema with the ERD. The use of an external API and ML integration were planned so that the experimental features could be easily isolated and replaced without interfacing the core app.

In essence, the system design now serves as a detailed roadmap for the next steps: code implementation of each requirement, testing of the corresponding functionalities, and gathering of user feedback. Each requirement is accounted for in the architecture, the separation of responsibilities is clear, and the platform is arranged to be scalable, testable, and extendable.

5. Testing and Evaluation

5.1 Introduction

This chapter describes the test and assessment plan for the OnChord platform.

The overall architecture consists of several independent but closely linked subsystems, including the frontend (React), backend API (Node.js + Express), database (Supabase/PostgreSQL), machine learning microservice (FastAPI), and external integrations (Spotify, Ticketmaster). Because of this layered structure, testing is planned at multiple levels that reflect the logical architecture of the system.

These tests are planned for the implementation phase and will be carried out once the core features are developed.

The aim of testing is to ensure that the system is:

- **Functionally correct** (meets the requirements)
- **Stable and performant**
- **Secure and reliable**
- **Usable and accessible**
- **Aligned with user expectations**
- **Scalable for future development**

Evaluation, on the other hand, focuses on assessing the **quality**, **effectiveness**, and **user experience** of the system based on real-world usage and experimental outcomes (especially for ML features).

Together, testing and evaluation will confirm whether the design decisions from Chapter 4 lead to a working, user-centred implementation of OnChord.

5.2 Plan for Testing

The following subsections describe the planned testing approach for each part of the system:

5.2.1 Unit Testing

Unit tests verify the correctness of individual components in isolation.
Relevant modules include:

Frontend (React)

- Component rendering
- Form inputs (review composer, playlist editor)
- State updates

- Error handling

Tools:
Jest, React Testing Library

Backend (Node.js + Express)

- Controllers (e.g., POST /review, GET /playlist/:id)
- Services (business logic)
- Middleware (authentication, validation)

Tools:
Jest, Supertest

Machine Learning Microservice (FastAPI)

- Taste similarity model input/output
- Playlist mood classifier input/output

Tools:
PyTest

5.2.2 Integration Testing

These tests ensure modules work correctly together.

Examples of Integration Tests

- Frontend → Backend → Database workflow for review creation
- Backend → ML microservice → Prediction response
- Backend → Spotify API → Data returned and formatted correctly
- Backend → Supabase (SQL queries, RLS policies)

Each test simulates real usage scenarios.

5.2.3 System Testing

System testing validates the whole system end-to-end.

Key system tests

Feature	Test Focus
Authentication	Successful OAuth login, token refresh failures
Review Creation	Draft saving, publishing, editing, deletion
Playlist Collaboration	Addition/removal of tracks, permissions
Insights Dashboard	Chart rendering, loading states, edge cases
Calendar & Reminders	Adding, editing, deleting reminders

Feature	Test Focus
Social Features	Follow/unfollow, visibility rules
External APIs	Handling rate limits, missing data, API failure

System testing ensures that all components behave correctly under real conditions.

5.2.4 User Acceptance Testing (UAT)

Once the key features are stable, users will be given access to a deployed version of OnChord.

UAT Process

1. Select testers (2-10 participants)
2. Assign tasks (e.g., create review, add playlist, view insights)
3. Record completion success/failure
4. Collect feedback (Google Form / observation)
5. Identify usability issues and improvements

This is essential for evaluating UX decisions made in the Figma design.

5.2.5 Structured Test Plan Table

Test ID	Component	Type	Description	Expected Result	Status
T01	Login	Integration	Spotify OAuth login	User logged in and token stored	Pending
T02	Review Service	Unit	Create review with valid data	Review saved to DB	Pending
T03	Playlist	System	Add track to collab playlist	Track appears for all collaborators	Pending
T04	ML API	Integration	Taste similarity prediction	Score returned < 1s	Pending
T05	Insights	System	Display listening stats	Graphs render correctly	Pending

This table shows initial planned test cases; additional tests will be added as more features are implemented

5.3 Plan for Evaluation

5.3.1 Usability Evaluation

Methods:

- Heuristic evaluation (based on Nielsen's 10 principles)

- User observations
- Task completion analysis
- Google Forms post-test questionnaire

Metrics:

- Time to complete tasks
- Error rates
- User satisfaction scores
- Accessibility considerations (dark mode, dyslexia-friendly layout)

5.3.2 Performance Evaluation

Backend

- API response time
- Database query performance
- ML inference latency

Frontend

- Load times (Vercel)
- Component rendering speed

Tools:
Chrome DevTools, Lighthouse, Supabase Logs, Render Metrics

5.3.3 Reliability & Error Handling

Evaluate how gracefully the system handles:

- Network failures
- API downtime (Spotify/Ticketmaster)
- Invalid inputs
- ML prediction failures

Logging from Render and Supabase will assist in fault analysis.

5.3.4 Machine Learning Evaluation

Machine learning models in this project are experimental.
Their effectiveness will be evaluated based on:

- Similarity score consistency
- User feedback (does the match “feel right”?)

- Manual testing with known examples

ML evaluation will not require extremely high statistical accuracy, as this is exploratory.

5.3.5 Requirements Evaluation

Evaluation will also check if the system meets its:

Functional Requirements

- Reviews
- Playlists
- Social features
- Insights
- Reminders
- Search

Non-Functional Requirements

- Performance
- Scalability
- Usability
- Security
- Reliability

A requirement traceability matrix can be added in the final report.

5.4 Conclusions

This chapter outlined a comprehensive strategy for verifying the correctness, performance, usability, and reliability of the OnChord platform. By combining multiple levels of testing, unit, integration, system, and acceptance, the testing plan ensures that each component functions individually and within the larger system.

Evaluation methods complement the testing process by assessing the user experience, system performance, and the effectiveness of the machine learning components. The mixed methodology of technical testing and human-centred evaluation reflects the dual nature of OnChord as both an interactive social platform and an experimental intelligent system.

These testing and evaluation plans will guide the validation phase during implementation and support the iterative refinement of the platform in later development stages.

6. System Prototype

6.1 Introduction

The goal of the prototype is to turn the design work from Figma into a working version of OnChord that can be used for testing and evaluation.
For the interim stage, the focus is on building the full front-end flow and wiring it to simple backend endpoints where needed (authentication, reviews, events, and insights).

The prototype will be built with:

- **React + TypeScript** for the user interface and routing.
- **React Router** for navigating between screens such as Home, Your Space, Insights, and Discover.
- A **component-based layout** that follows the structure of the Figma designs.
- A simple **API layer** (for now using mocked or test endpoints) that can later be swapped for the real Node.js/Express backend and ML microservice.

The screenshots below show the main user journeys that the prototype will implement:

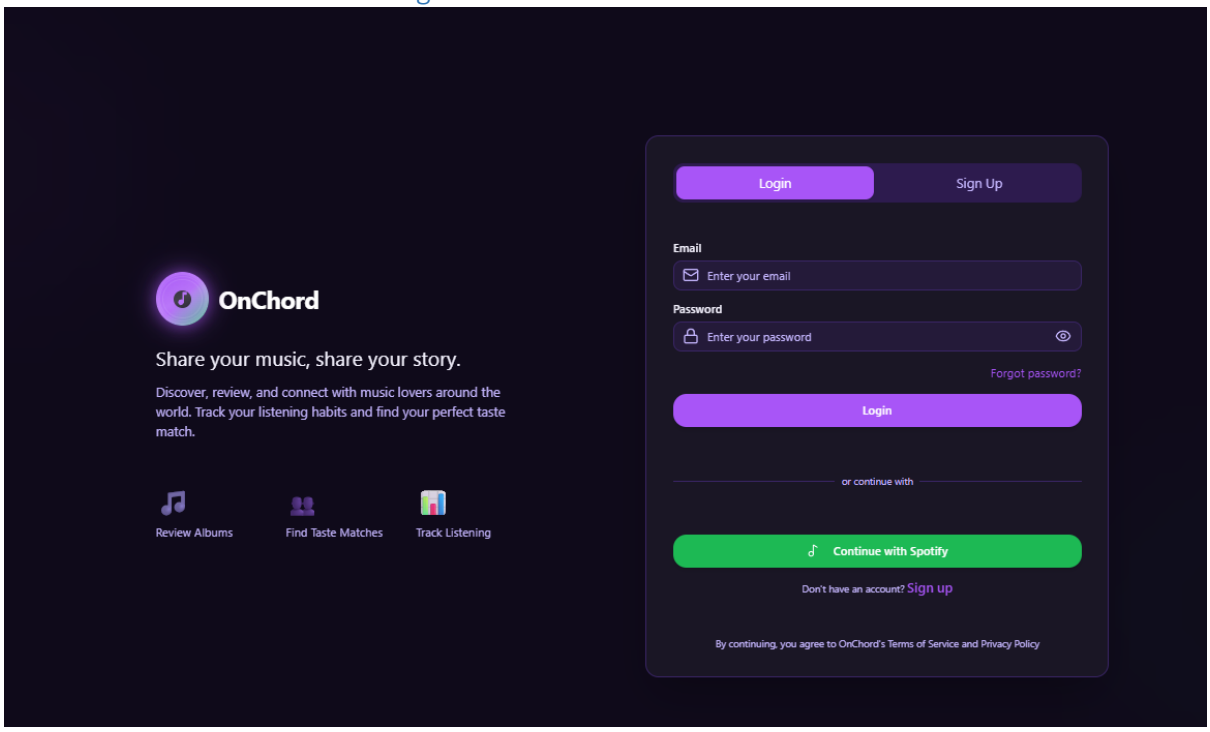
- Logging in and signing up (including "Continue with Spotify").
- Browsing and writing reviews from the homepage.
- Managing profile, stats, lists, and favourites in "Your Space".
- Viewing concerts and adding reminders.
- Reading and sending messages.
- Browsing reviews using a calendar view.
- Viewing "Taste Match" results and "Mood Analysis" for music insights.
- Discovering new albums and songs.
- Viewing in-app notifications.

These journeys map directly to the core use cases from Chapter 3 and will be used later for testing (Chapter 5).

6.2 Prototype Development

This section describes how the Figma screens represent the first version of the system and how each major UI area will be developed in code.

6.2.1 Authentication and Landing Screen

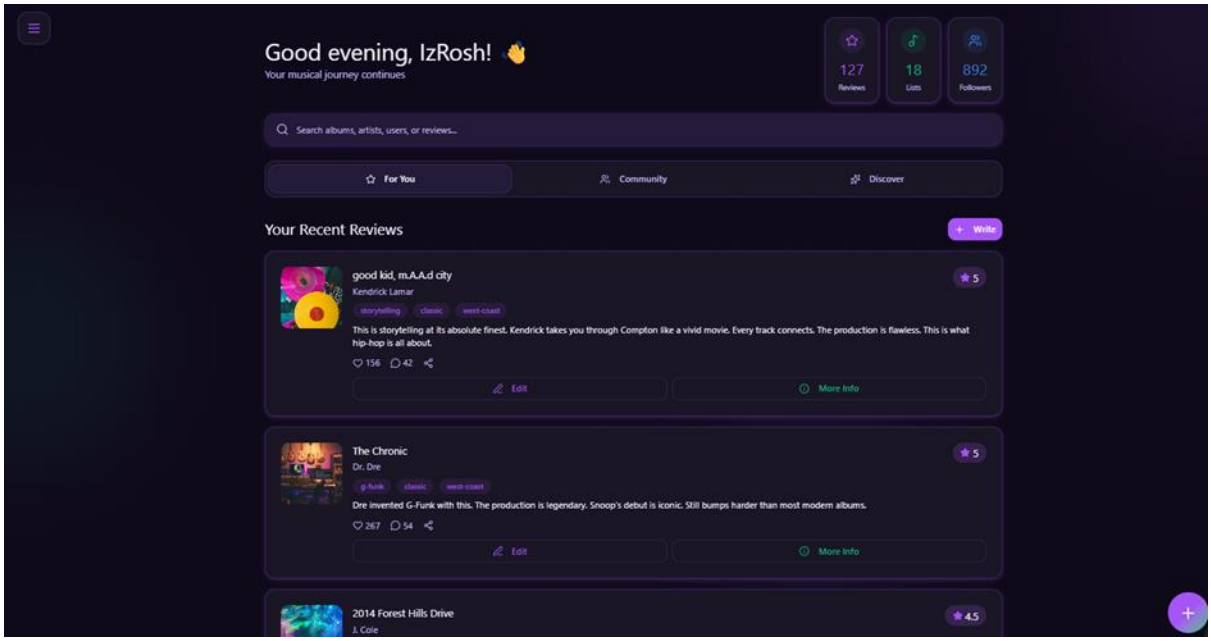


The landing screen combines marketing content ("Share your music, share your story") with the login and sign-up form. In code this will be implemented as:

- A top-level **AuthLayout** component that splits the screen into a left marketing panel and a right authentication card.
- A **LoginForm** component with controlled inputs for email and password, client-side validation, and error states.
- A **tab switcher** at the top (Login / Sign Up) controlled by local component state.
- A **SpotifyAuthButton** component that triggers the OAuth flow by redirecting to the backend /auth/spotify endpoint.

When the user logs in successfully, the frontend will store a short-lived auth token in memory (and possibly refresh token in secure storage) and navigate to the **home** route.

6.2.2 Home Feed and Recent Reviews



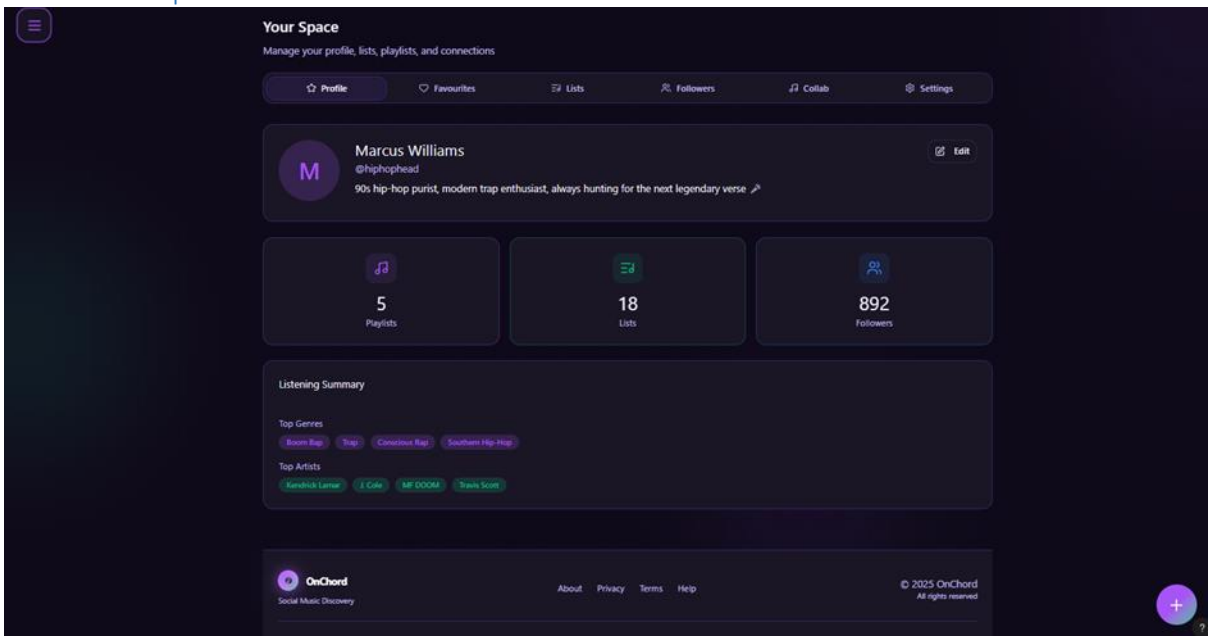
The home page is the main hub for browsing content. React will implement this with:

- A **HomePage** container tied to the /home route.
- A **SearchBar** component at the top which will later be connected to a search endpoint for albums, artists, users, and reviews.
- A **tab strip** ("For You / Community / Discover") that controls which feed is loaded. This will be represented as local state plus a query parameter so it can be bookmarked.
- A **ReviewList** component that renders a list of **ReviewCard** components. Each card shows artwork, album metadata, tags, like / comment / share counts, and an Edit button.

In the prototype, ReviewList will first load dummy data from a JSON fixture. Once the backend is ready, it will call an endpoint like GET /feed/recent and map the results into the same card format.

The floating "+" button and "Write" button will route the user to a dedicated **ReviewComposer** page.

6.2.3 “Your Space” - Profile and Stats

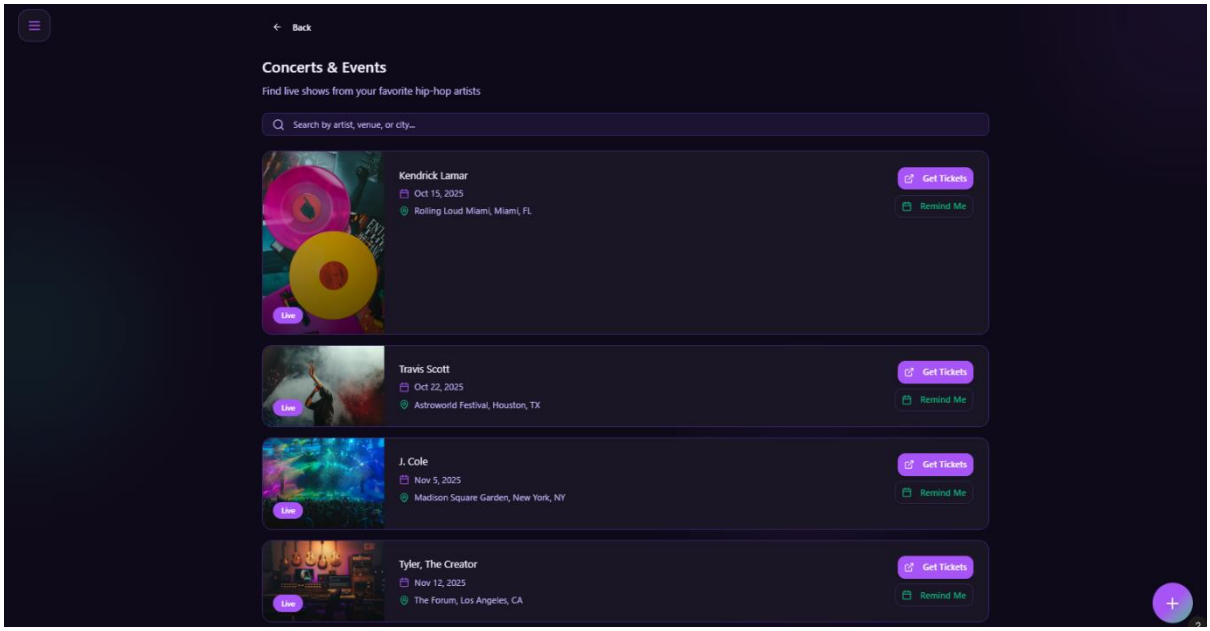


The “Your Space” screen acts as the user dashboard. In React it will be built as:

- A **UserSpacePage** component with nested tabs (“Profile / Favourites / Lists / Followers / Collab / Settings”).
- A **ProfileHeader** component that displays the user avatar, handle, bio, and an Edit button.
- A **StatsRow** component that shows quick metrics such as number of playlists, lists, followers, and basic listening summary.

Each tab will be implemented as a child route (e.g. /space/profile, /space/favourites) so the URL always reflects what the user is viewing. For the interim prototype, fake stats will be generated from fixtures; later these will have loaded from /me and /me/stats endpoints.

6.2.4 Concerts & Events

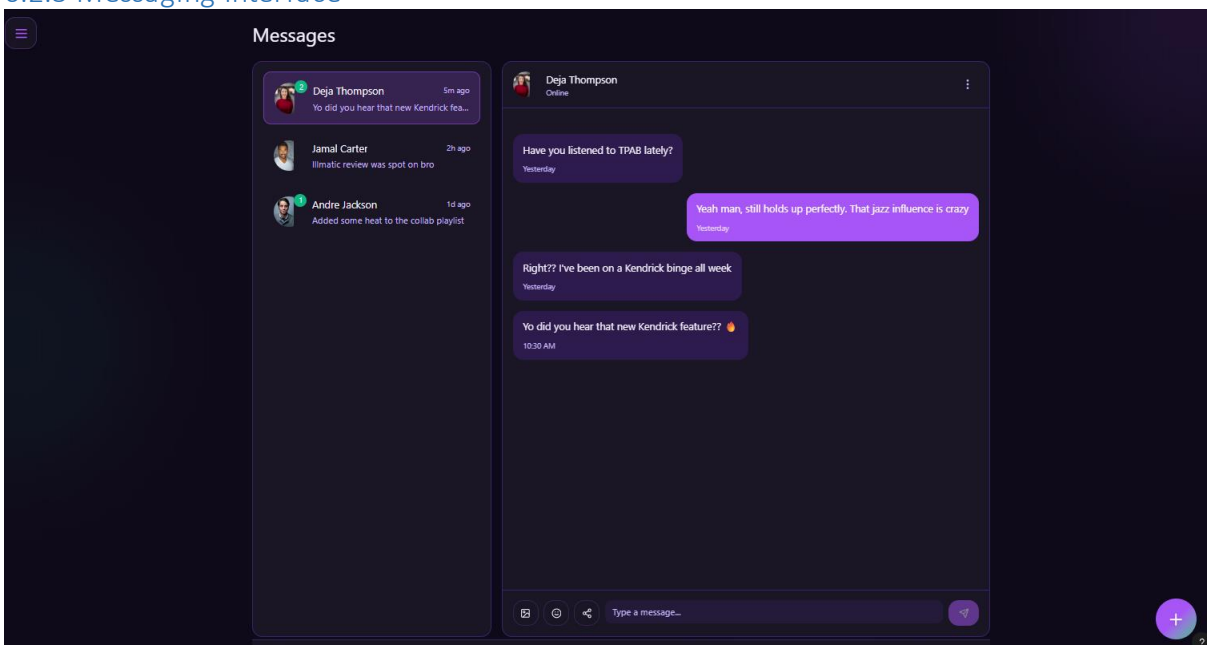


The events screen lets users search for live shows and set reminders. Code-wise:

- A **EventsPage** component will provide the layout and search bar.
- A **EventCard** component will display artwork, artist name, date, venue, and actions ("Get Tickets", "Remind Me").
- The search bar will call a simple function that, in the prototype stage, filters a local array of events. In the full system, it will call the backend, which in turn talks to the Ticketmaster API.

Clicking **"Remind Me"** will later trigger a POST to /reminders, but in the interim prototype it will at least update UI state so the button appears "active" and can be demonstrated in UAT.

6.2.5 Messaging Interface

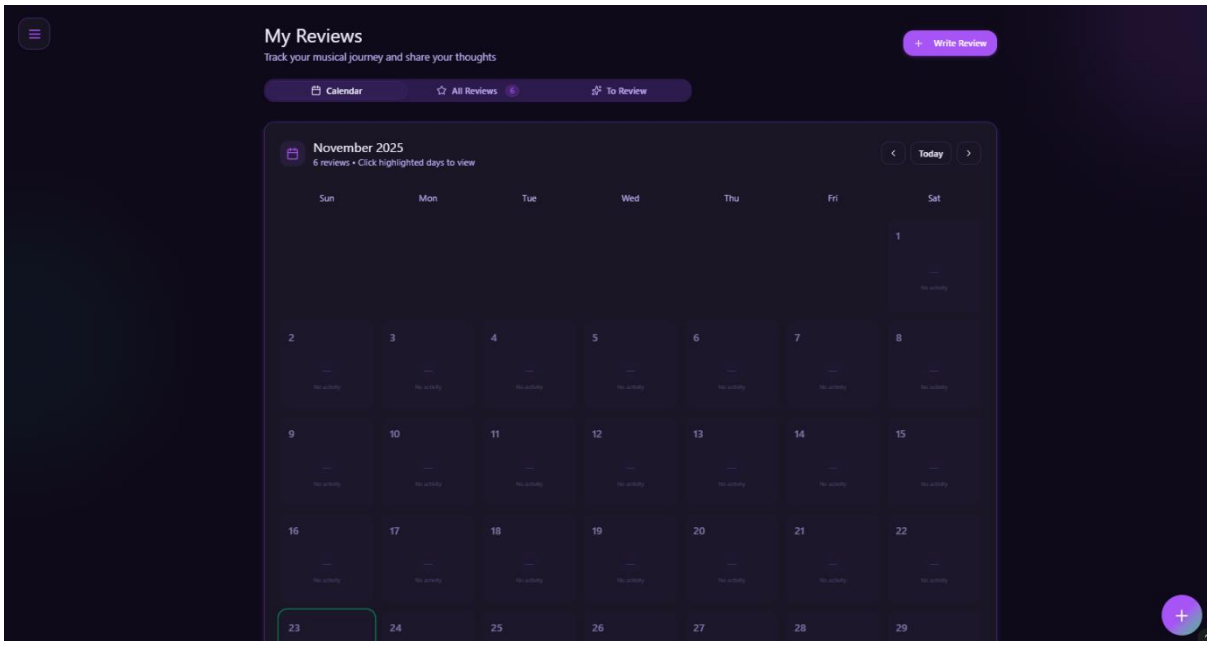


The messaging UI is structured similarly to common chat apps. The prototype will include:

- A **MessagesPage** with a two-column layout (conversations list on the left, active conversation on the right).
- A **ConversationList** component that shows each chat with unread count and last message snippet.
- A **ChatWindow** component that renders message bubbles, timestamps, and typing area.
- A **MessageInput** component handling keyboard shortcuts (Enter to send) and basic validation.

For the interim prototype, messages can be backed by a local array or mock API. Later, this structure can be swapped to real-time messaging using Supabase or websockets without changing the UI components.

6.2.6 Review Calendar View

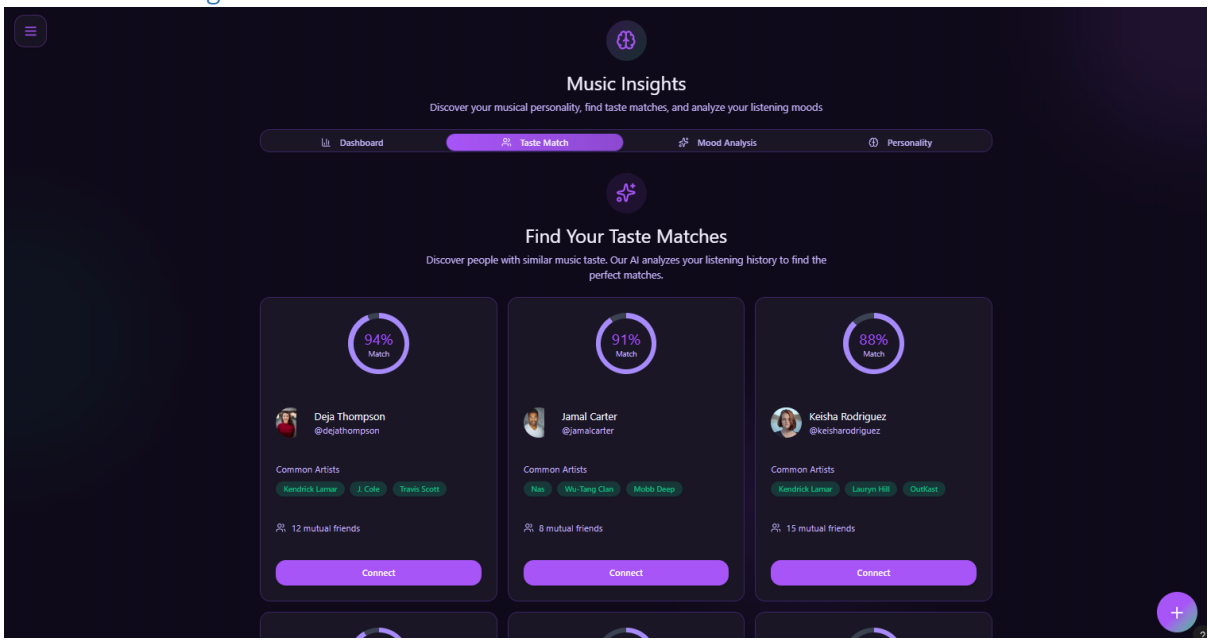


The calendar screen shows when the user has written reviews and lets them jump to those entries. Implementation details:

- A **ReviewCalendarPage** that fetches a list of "reviewed dates" and passes them into a calendar component.
- A **CalendarGrid** component (either custom or based on a small date-utility library) that renders days for the current month and highlights days with activity.
- A **mode switcher** at the top ("Calendar / All Reviews / To Review") that controls whether the calendar or list view is shown.

When a date is clicked, the prototype will open a side panel or modal showing the reviews from that day, using the same ReviewCard component for consistency.

6.2.7 Music Insights - Taste Match

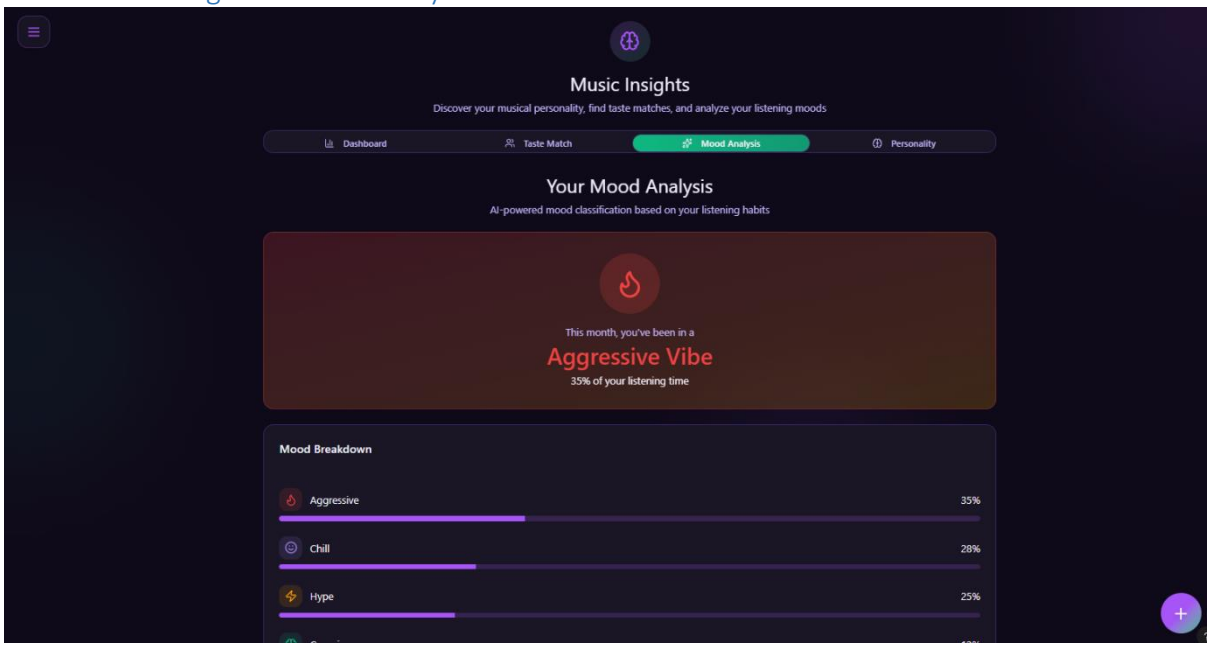


The Taste Match tab visualises similarity scores between the user and other people. The React implementation will include:

- An **InsightsPage** route with nested tabs ("Dashboard / Taste Match / Mood Analysis / Personality").
- A **TasteMatchTab** component that calls an endpoint like GET /insights/taste-match (mocked during early development) and displays the results in **MatchCard** components.
- Each MatchCard shows:
  - A circular progress indicator (e.g. 94% Match).
  - The other user's avatar, name, handle.
  - Common artists and mutual friends.
  - A **Connect** button that reuses the follow logic from the Social Interaction use case.

For now, similarity scores can be generated from static data; later they will come from the ML microservice via the backend.

6.2.8 Music Insights - Mood Analysis

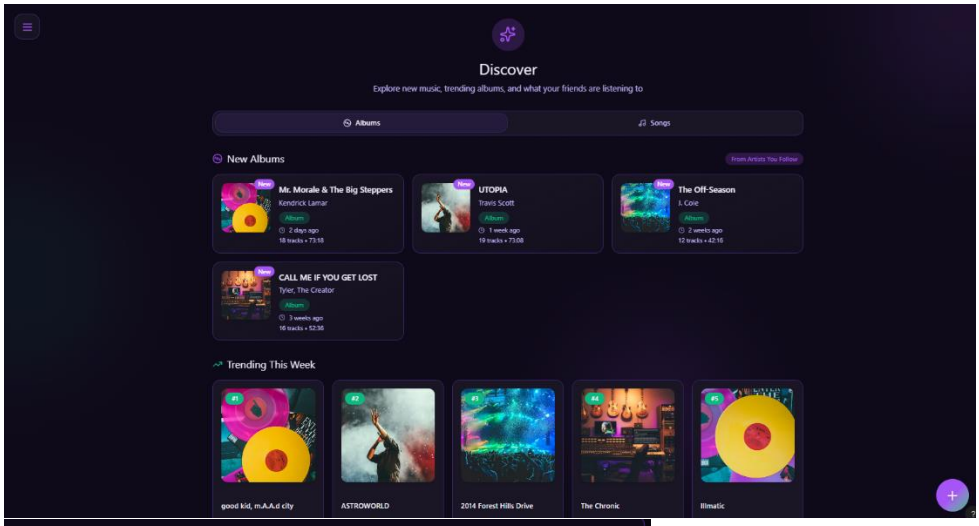


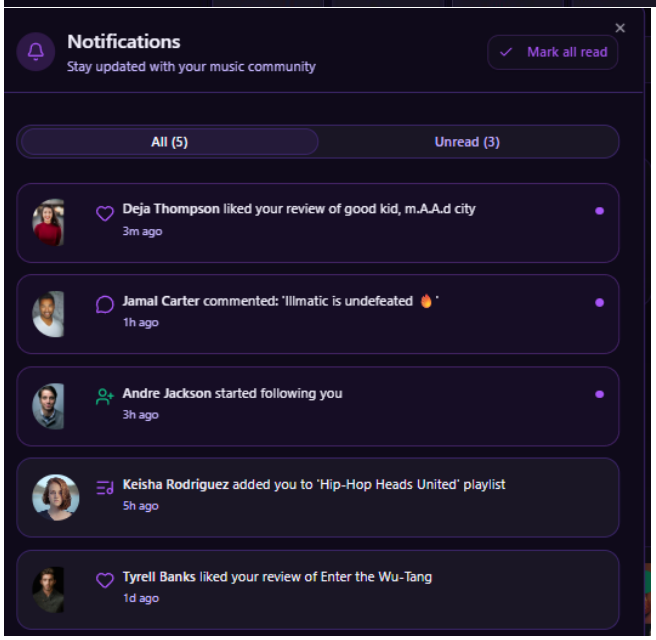
The Mood Analysis tab shows an overall mood label (e.g. “Aggressive Vibe”) and a breakdown of listening moods. Implementation details:

- A **MoodAnalysisTab** component under the Insights page.
- A **MoodSummaryCard** that displays the main monthly mood and percentage.
- A **MoodBreakdownList** using horizontal progress bars for Aggressive, Chill, Hype, etc.

When wired to the backend, this component will consume an endpoint such as GET /insights/mood, which will in turn call the playlist mood classifier in the ML microservice.

6.2.9 Discover & Notifications





The Discover page will be implemented with:

- A **DiscoverPage** route that fetches “New Albums”, “Trending This Week”, and separate tabs for Albums vs Songs.
- A reusable **MediaCard** component used across Discover, Home, and lists.

The Notifications panel will be a global component:

- A **NotificationsDrawer** that slides down / in from the top when the bell icon is clicked.
- A **NotificationItem** component for each event (likes, comments, follows, playlist invites).
- Tabs for “All” vs “Unread”, implemented with simple state filters.

Initially, notifications can be mocked; later they will be fetched from /notifications and updated after key actions (like, comment, follow).

6.3 Results

This section links the prototype to the tests planned in **Section 5**. It explains, in plain terms, what I expect to check once the code is built.

- **Login screen**
  - Can a user log in successfully?
  - Do wrong details give clear error messages?
- **Reviews on the home page**
  - Can a user write, edit, and delete a review?
  - Does the new review show up in "Your Recent Reviews"?
- **Playlists and profile area**
  - Can users see their playlists, and basic stats?
  - When they change something in a playlist, does the screen update?
- **Concerts & events**
  - Do event cards load correctly?
  - Does "Remind Me" create a reminder once the backend is wired?
- **Messages**
  - Do conversations load?
  - When a message is sent, does it appear in the thread?
- **Review calendar**
  - Are days with reviews highlighted?
  - When a day is clicked, are the right reviews shown?
- **Taste Match**
  - Do match cards load with a % score and shared artists?
  - If the ML service is down, does the page show a friendly "not available" message instead of breaking?
- **Mood Analysis**
  - Does the main mood label and percentage match the data returned by the model?
  - Do the mood bars add up and look reasonable?
- **Discover + notifications**
  - Does the Discover page load albums without slowing everything down?
  - Do the notifications panel open and list new activity?

For **user testing (UAT)**, I expect a small group of users to try simple tasks (log in, write a review, find a taste match, set a reminder). If most people can do these without help and without errors, the prototype will be considered acceptable for this stage.

6.4 Evaluation

The prototype will be evaluated using the methods planned in **Section 5**, but in a light, practical way.

6.4.1 Usability & UX

- Ask a small group of users to: log in, write a review, make a playlist, and view their insights.
- Watch where they hesitate or get stuck and time how long tasks take.
- Collect quick feedback with a short Google Form about ease of use, look and feel, and what feels confusing.

6.4.2 Performance

- Use browser tools to check how fast key screens load (login, home, profile, insights).
- Note anything that feels slow, such as scrolling large lists or loading charts, and link these to specific API calls or components to optimise later.

6.4.3 Reliability & Error Handling

- Turn off the network or force fake API errors to see what happens.
- Check that the app shows clear messages (e.g. "Unable to load data, please try again") instead of crashing and always lets the user recover.

6.4.4 Machine Learning Behaviour

- Try the taste match and mood pages with a few test users and playlists where the "right" answer is roughly known.
- Ask testers if the scores and moods feel believable, not perfect.
- Note any obvious mistakes as ideas for future model tuning.

6.4.5 Requirements Check

- Compare the working prototype against the main requirements from Chapter 3:
    - core flows (login, reviews, playlists, insights, events, social)
    - non-functional points (dark accessible UI, basic performance, clear errors).
- Any gaps found here will feed directly into the next development sprint.

6.5 Conclusions

This chapter described the first working prototype of OnChord and how it will be tested and evaluated.

- The **Figma designs** were translated into a clear frontend plan, covering the main user journeys: onboarding, home feed, reviews, insights, events, and notifications.
- The **prototype plan** is aligned with the logical architecture from Chapter 4, so each screen maps cleanly to React components, backend routes, database tables, and (where needed) ML endpoints.
- A **testing and evaluation strategy** has been set up that looks at four things:
    - does each feature work,

- is the app fast enough,
- is it stable when things go wrong,
- and is it enjoyable to use.

The result of this stage will be a functioning early version of OnChord that is capable of being demonstrated to real users, employed for collecting feedback, and further developed during the last development phase. The following actions are to finalize the implementation, conduct the tests as outlined, and, by means of the evaluation results, enhance both the code and the user experience.

7. Issues and Future Work

7.1 Introduction

This section looks at the **problems, risks, and open questions** in the OnChord project, and how they might affect the final system. It also outlines what still needs to be done and how I plan to complete the work within the remaining time.

Because this is an experimental project that mixes a full-stack web app with basic machine learning, it is expected that not everything will go perfectly. The aim here is to be honest about the issues and show a clear plan for how to move forward.

7.2 Issues and Risks

Based on the current design, prototype, and planned evaluation, the main issues and risks are:

7.2.1 Integration with External APIs

OnChord depends on services like **Spotify** and **Ticketmaster**. There are risks such as:

- API rate limits or access restrictions.
- Changes to the API during development.
- Authentication complexity with OAuth.

If these APIs are slow, unavailable, or restricted, some features (like live listening stats, event listings, or rich metadata) may not work as planned.

7.2.2 Machine Learning Quality and Scope

The machine learning parts of the project (taste matching and playlist mood classification) are **experimental**. Risks include:

- Difficulty finding suitable datasets or features.
- Models that give weak or inconsistent results.
- Time needed to train, test, and tune models.

There is a chance that the ML features will work only at a basic level, or that they must be simplified to fit the project timeline.

7.2.3 Time and Complexity

OnChord is a **full-stack system** with many moving parts:

- React frontend

- Node/Express backend
- Supabase/PostgreSQL
- ML microservice
- Multiple APIs

As a single developer, this is a lot to build, test, and polish. The risk is that some areas (for example messaging, notifications, or advanced social features) may need to be cut back or left as partial implementations so that the core experience is stable and complete.

7.2.4 Performance and Reliability

Potential issues include:

- Slow loading of large feeds (reviews, discover lists).
- Slow responses from the ML service.
- Errors when external APIs are down.

If not handled carefully, this could hurt the user experience and make the app feel unreliable.

7.2.5 UX and Accessibility

The Figma designs and prototype aim to be clear and accessible, but risks remain:

- Some flows might still be confusing in practice (for example, navigating between Your Space, Insights, and Discover).
- Important actions (like editing reviews or using the calendar) might not be obvious to first-time users.
- Accessibility issues such as colour contrast or font size might need more work.

7.2.6 Conclusion

Overall, I do hope to achieve as many aspects as possible for this project, familiarising myself with new technologies while also further implementing old technologies that I'm experienced with will be difficult, but in the long term will be more enjoyable than anything. For the sake of possibly even using this as a startup platform it's within my best interest to use the time I'm given to really focus on creating the best output I can for the future

7.3 Plans and Future Work

To manage these risks and complete the project, the plan is to:

7.3.1 Prioritise Core Features

Focus first on making sure the **core flows are fully working**, even if some extra features stay simple:

- User login and basic account handling.
- Creating, editing, and viewing reviews.
- Creating playlists/lists and managing favourites.
- Viewing taste match and mood analysis (even if the ML behind it is simple at first).

Secondary features, such as messaging, advanced notifications, or very detailed stats, can be scaled down if time becomes tight.

7.3.2 Phase the Machine Learning Work

Instead of trying to build a perfect ML system, the plan is:

1. Start with **dummy ML responses** so the UI and flows can be built and tested.
2. Replace these with **basic models** (for example, simple similarity based on genre tags or track features).
3. If time allows, improve the models and add better evaluation.

If ML is delayed, the app will still work; the ML outputs will just be less intelligent and clearly described as experimental.

7.3.3 Handle API Risks

To deal with API uncertainty:

- Use **mock data** during early development so the app is not blocked if Spotify or Ticketmaster are unavailable.
- Wrap all external calls in **backend helpers**, so if an API changes, only one part of the code needs to be updated.
- Show clear messages in the UI ("Spotify is currently unavailable") instead of failing silently.

If necessary, some features can fall back to cached or static data instead of real-time data.

7.3.4 Improve UX Iteratively

As soon as a basic version of the app is running:

- Run small **user tests** with classmates or friends.
- Ask them to perform simple tasks and note where they get lost.
- Use this feedback to adjust navigation, labels, and button placement in both Figma and the code.

Accessibility will also be checked against basic guidelines (contrast, text size, clear icons).

7.3.5 Documentation and Reflection

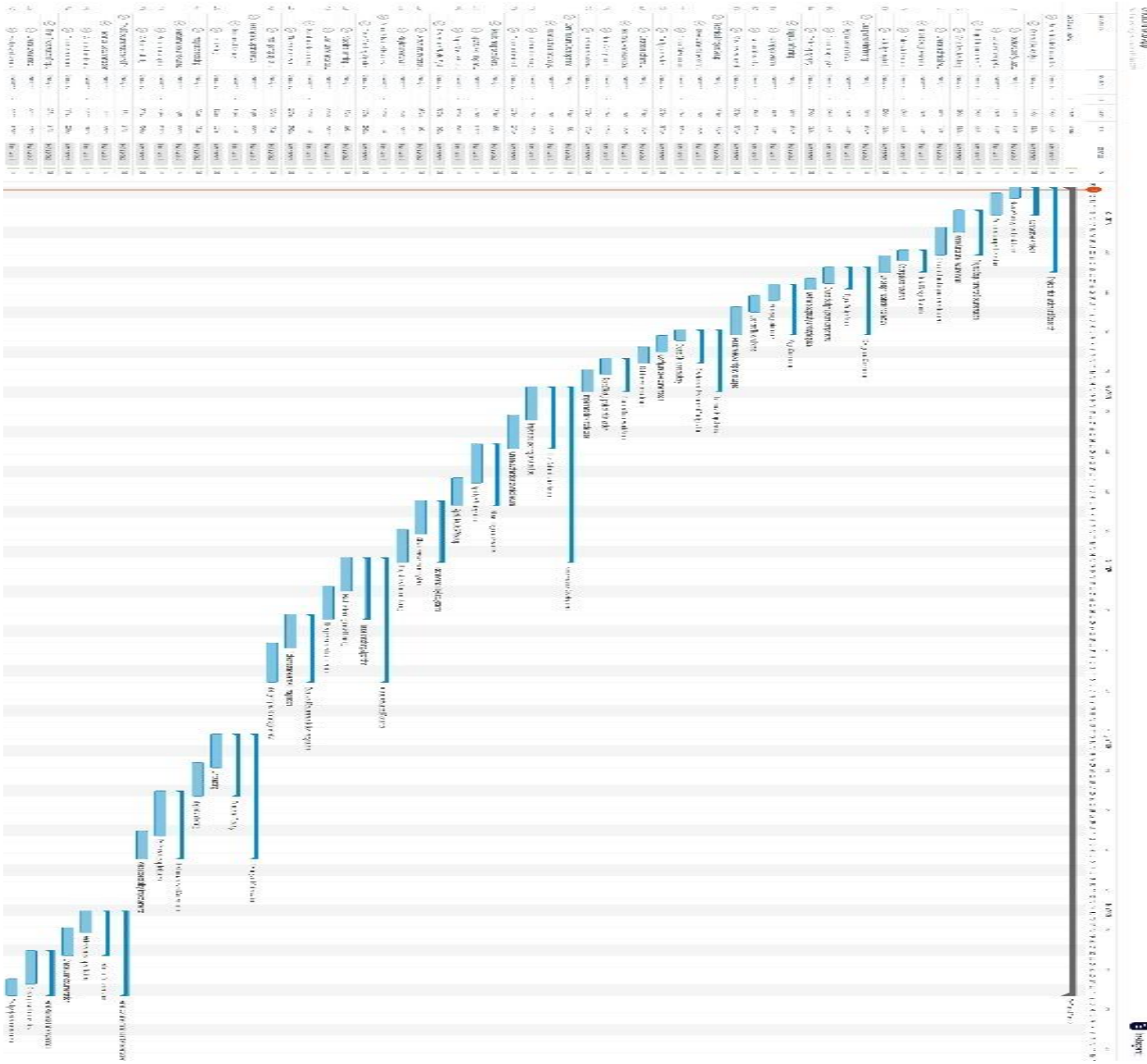
Throughout development:

- Keep notes of what worked, what struggled, and any changes of plan.
- Document limitations honestly (for example, ML accuracy or incomplete features).

This is important because the project is framed as an **experiment**. Even if everything is not perfect, a clear explanation of what was tried, why it failed or succeeded, and what would be done next still makes for a strong final report.

7.3.1 Project Plan with GANTT Chart

Phase	Timeframe	Main Deliverables	Short Description
Phase 1 - Core Backend & DB	Weeks 1-3	Stable API + Supabase schema	Finalise database tables, and implement core Node/Express routes for auth, reviews, playlists, follows, favourites, and basic validation.
Phase 2 - Core Frontend MVP	Weeks 3-6	Working React interface	Build login, home feed, review editor, playlists page, and profile page, and connect them to the live API so the main user journey works end-to-end.
Phase 3 - Insights & ML	Weeks 6-8	Taste Match + Mood Analysis prototype	Implement the FastAPI microservice, simple taste-match and mood algorithms, and connect them to the **Music Insights** screens.
Phase 4 - Testing & Evaluation	Weeks 8-10	Test results + user feedback	Run the tests listed in Chapter 5 (unit, integration, basic performance, small user study). Fix major bugs and record results.
Phase 5 - Polish & Accessibility	Weeks 10-11	UI refinements + accessibility fixes	Improve styling, empty states, error messages, and check contrast, keyboard use, and general usability.
Phase 6 - Final Report & Submission	Weeks 11-12	Final dissertation & artefacts	Update all chapters, insert final screenshots and diagrams, add reflection on limitations and future work, and prepare the final submission package.



This phased plan replaces the original high-level Gantt view and shows how development, testing, and documentation will be scheduled. The exact dates may shift slightly, but the focus is to deliver a working experimental version of OnChord with core features, ML demos, and a complete written report.

References

Hint:

Use Zotero to manage your references (see Brightspace resources).

Use the **Harvard** or **IEEE** numbering referencing style

- https://www.zotero.org/support/quick_start_guide

1. Music Business sources: https://www.musicbusinessworldwide.com/global-recorded-music-revenues-hit-296bn-in-2024-up-4-8-yoy-users-of-paid-music-subscriptions-reach-752m/
2. Music streaming side-effects: https://www.weforum.org/stories/2023/03/charted-the-impact-of-streaming-on-themusic-industry/
3. Need for social music platforms: https://www.bbctech.co.uk/entertainment/the-evolution-of-music-streaming-and-itsimpact-on-the-music-industry/
4. Letterboxd's success: https://www.forbes.com/sites/paultassi/2023/02/04/why-letterboxd-is-thriving-in2023/
5. Music recommendations algorithms: https://www.theguardian.com/technology/2025/jun/18/up-to-70-of-streams-of-aigenerated-music-on-deezer-are-fraudulent-says-report
6. arXiv Academic research: https://arxiv.org/abs/2405.14999
7. **Global streaming growth (+4.8%) and 752M paid subscribers**
   Music Business Worldwide:
   https://www.musicbusinessworldwide.com/global-recorded-music-revenues-hit-29-6bn-in-2024-up-4-8-yoy-users-of-paid-music-subscriptions-reach-752m/
8. **Streaming has reshaped listening behaviour**
   World Economic Forum:
   https://www.weforum.org/stories/2023/03/charted-the-impact-of-streaming-on-the-music-industry/
9. **Algorithmic ecosystems lack transparency; rise of AI issues**
   The Guardian:
   https://www.theguardian.com/technology/2025/jun/18/up-to-70-of-streams-of-ai-generated-music-on-deezer-are-fraudulent-says-report

Appendix A: System Model and Analysis

Details of Requirements gathering and analysis method

A.1 Overview

This appendix summarises how the requirements for OnChord were gathered, analysed, and turned into an initial system model. It supports Chapter 3 by giving extra detail on the techniques used and how they fed into the final architecture and use cases.

A.2 Requirements Gathering Methods

A mix of qualitative and quantitative methods were used:

1. User Survey (Google Forms)

- Target group: students and casual music listeners.
- Topics:
  - How often they review or rate music
  - Interest in playlists, favourites, and social features
  - Desire for statistics, insights, and "taste match" features
  - UI preferences (dark mode, layout, complexity)
- Main outcomes:
  - Strong interest in review journaling and visibility controls
  - High demand for personalised insights (stats, taste match, moods)
  - Preference for a simple, visually clean dark interface

OnChord Questionnaire (Social Music Discovery & Journaling Platform)

When you submit this form, it will not automatically collect your personal details such as your name or email address unless you choose to provide them.
This form is part of a final-year research project for the TU857 BSc in Computer Science (Infrastructure) at TU Dublin.

The project, conducted by **Emmanuel Awode** (C22320583@mytudublin.ie), investigates how music listeners engage with digital platforms to discover new music, review albums, and connect with others who share similar music tastes.
The system being developed — *OnChord* — is a social music journaling web application that uses data insights and lightweight AI models to help users understand and share their listening habits.

The information collected through this questionnaire will help identify user preferences, guide feature development, and support usability research for the OnChord platform.
Privacy and Data Use Statement

All information collected through this form will be used **solely for academic research purposes** related to this project.
Participation is **completely voluntary**, and all responses will be treated confidentially.
No personally identifiable information will be recorded unless you explicitly provide it.

All data will be securely stored and accessible only to the researcher and within the context of TU Dublin's FYP evaluation.
Your responses will **not** be shared with third parties or used for commercial purposes.

Participants may withdraw at any time before submitting the form. Once submitted, responses will be anonymized and cannot be traced back to individual contributors.

This project has been reviewed and is conducted in accordance with **TU Dublin's ethical research guidelines**.

If you do not wish to give consent, please close the form now.

* Indicates required question

1. Consent *

- ◯ I have read and understood the above statement and consent to my data being used for the purposes of this research project.

What platform do you use most for listening to music? *

- ◯ Spotify
- ◯ Apple Music
- ◯ YouTube Music
- ◯ SoundCloud
- ◯ Other

How do you usually discover new music? *(Select all that apply)* *

- ☐ Recommendations from friends
- ☐ Social media (TikTok, Instagram, etc.)
- ☐ Music streaming app suggestions
- ☐ Playlists or radio
- ☐ Online forums or communities
- ☐ Other

Do you currently use any apps or websites to share your music opinions or reviews? *

- ◯ Yes
- ◯ No

Would you be interested in using a platform where you can write short reviews or thoughts about songs and albums? *

- Definitely
- Maybe
- Not really

How interested are you in seeing what your friends are listening to or reviewing? *

- Very interested
- Somewhat interested
- Not interested

How important is it to you to connect with people who have a similar music taste? *

- Very important
- Somewhat important
- Not important

Would you use a feature that recommends new friends or playlists based on your listening habits? *

- Yes
- Maybe
- No

Would you use collaborative playlists (where you and others can add songs together)? *

- Yes
- Maybe
- No

Back Next Clear form

Rating features

How appealing do you find features like:

- Viewing your **music stats** (hours listened, top artists, etc.)
- Having a **calendar** of your music reviews
- Seeing **local concert recommendations**

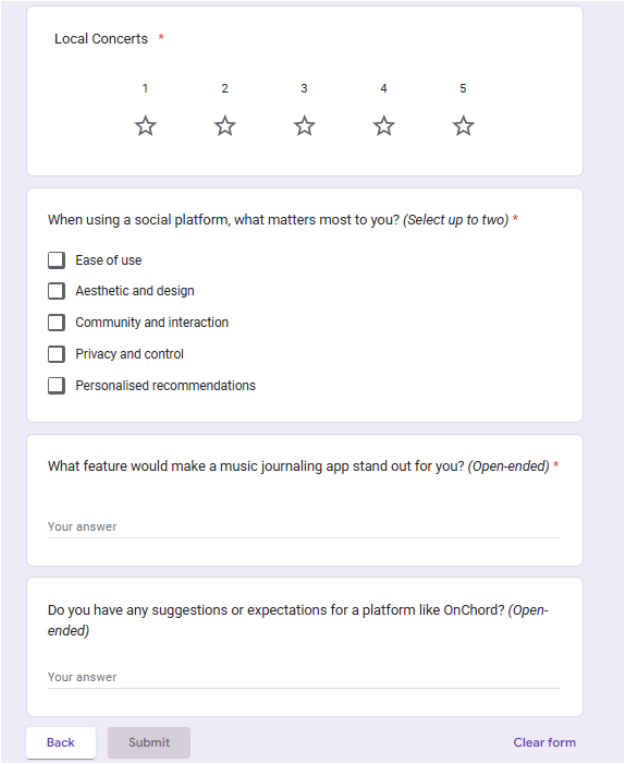
(Rate each from 1–5, 1 = Not Useful and 5 = Very Useful

Music Stats *

1 2 3 4 5

Review Calendar *

1 2 3 4 5



2. Competitor Analysis

- Analysed: Letterboxd, Last.fm, Stats.fm, RateYourMusic, Spotify Wrapped.
- Looked at: feature sets, interaction patterns, strengths/weaknesses.
- Outcomes:
  - Confirmed demand for review-style logging and social discovery
  - Identified a gap for combined journaling + social + light ML insights
  - Inspired features such as review feeds, profile pages, and insights dashboards

3. Figma Prototype Review

- Early wireframes and high-fidelity screens were created for:
  - Login / onboarding
  - Home feed and review cards
  - Profile ("Your Space"), favourites and lists
  - Music Insights (Dashboard, Taste Match, Mood Analysis)
  - Events, messages, notifications, calendar views
- Outcomes:
  - Helped identify required navigation structure and routes
  - Revealed all key UI states (empty, loading, error)
  - Supported refinement of use cases (what users do on each screen)

4. Academic & Industry Research

- Readings on recommender systems, collaborative filtering, and mood classification.

- UX and HCI sources on dark-mode design, accessibility, and social platforms.
- Outcomes:
    - Informed the choice of simple, explainable ML models (e.g. similarity scores, basic mood buckets)
    - Reinforced the need for transparency and not over-promising ML accuracy

A.3 Requirements Analysis Approach

Instead of using MoSCoW, requirements were split into:

- **Functional requirements** - what the system must do
    - e.g. write/edit reviews, manage playlists, follow/unfollow, view insights, set reminders.
- **Non-functional requirements** - how the system behaves
    - e.g. performance targets, security (OAuth), usability, reliability, scalability.

These were then mapped to:

- Use case narratives
- Data entities
- System components

A.4 System Model Artefacts

The following artefacts were produced as part of the system analysis:

- **Use Case Diagram** - shows User and Friend User interacting with:
    - Manage Reviews
    - Manage Playlists & Lists
    - Social Interaction
    - View Insights & Discover Music
    - View Public Content
    - Interact Socially
    - Follow / Unfollow User
- **Use Case Narratives -** step-by-step descriptions of each high-level interaction, including main flow, alternative flows, and basic error handling.
- **Conceptual ERD / Data Model** - initial entity list and relationships, later refined into the Supabase schema in Chapter 4.
- **Initial Architecture View** - four-layer view (Frontend, Backend API, Database, ML microservice) that later evolved into the logical and physical architecture diagrams.

A.5 Summary

In summary, the system model and requirements were not guessed in isolation; they emerged from:

- real user feedback,
- analysis of existing platforms,
- design experiments in Figma, and
- relevant academic/industry work.

This gave a grounded basis for the design choices in Chapter 4 and ensured that the prototype focuses on realistic, achievable features that users care about.

Appendix B: Design

Details of design approach used

System elements, components, classes

B.1 Design Approach

The system uses a **layered, component-based** design:

- Start from user journeys and Figma screens.
- Split the system into frontend, backend, database, and ML microservice.
- Keep each part small, reusable, and focused on a single responsibility.

B.2 Frontend Design (React + TypeScript)

The frontend is divided into **pages** and **reusable components**.

Main pages (examples):

- Login / Onboarding
- Home feed (reviews, tabs for For You / Community / Discover)
- Profile ("Your Space")
- Music Insights (Dashboard, Taste Match, Mood Analysis)
- Events / Reminders
- Messages and Notifications
- Review Calendar
- Discover

Key reusable components:

- Layout: navbar, sidebar, page headers
- Content: review cards, album cards, event cards, user cards
- Inputs: search bar, rating stars, toggles, text fields
- Insights: match gauge, mood bars, stats panels
- Feedback: loading, empty states, error banners

All follow the same dark-theme style from Figma.

B.3 Backend Design (Node.js + Express)

The backend is organised into **routes, controllers, and services**.

Main service areas:

- Auth service - login, Spotify OAuth, sessions
- Review service - create/edit/delete reviews, visibility rules
- Playlist service - playlists and collaborative editing
- Social service - follow/unfollow, follower lists
- Insights service - fetch stats, call ML microservice
- Events service - concerts and reminders

Each service talks to Supabase and returns clean JSON to the frontend.

B.4 Data Model (Supabase / PostgreSQL)

Core entities (tables) include:

- users
- reviews
- playlists and playlist_tracks
- lists and list_items
- favourites
- follows (social graph)
- likes and comments
- reminders
- events_cached

Relationships follow the ERD from Chapter 4 (e.g. one user → many reviews, many-to-many follows).

B.5 Machine Learning Microservice (FastAPI + Python)

The ML part is kept small and separate:

- POST /predict/taste_similarity - returns a similarity score between two users.
- POST /predict/mood_classification - returns a mood label and breakdown for a playlist.

The main backend just sends JSON to this service and receives predictions back.

B.6 Summary

In short, OnChord is designed as:

- A React frontend made of reusable components
- A Node/Express backend with clear service modules

- A structured relational database in Supabase
- A small Python microservice for experimental ML insights

This design makes the system easier to build, test, and extend in later work.

Appendix C: Prompts Used with ChatGPT

C.1 Project Idea and Scope

- *"I want to make a full stack web app based on music, with a feasible combination of the apps: Letterboxd, StatsFM, Turn Up and Tinder. Any ideas you may have for this? This would be ideally for a 4th year college project."*
- *"What kind of machine learning features would be feasible for this type of music/social app?"*

C.2 Name, Branding and Colour Scheme

- *"What names can I use for a music web app related to listening, sharing or chords?"*
- *"Suggest colour palettes for a calm, dark, chill music app with purple, mint and pink accents."*

C.3 Requirements, Use Cases and System Model

- *"Guide me through high-level use cases for a user who can write reviews, create playlists, view insights, and follow other users."*
- *"Explain how to turn my functional and non-functional requirements into an initial systems model for the report."*

C.4 Architecture, ERD and Diagrams

- *"Help me design a logical architecture for OnChord with a React frontend, Node/Express backend, Supabase database and a FastAPI ML microservice."*
- *"What's a good website to creative simple ERD's?"*

C.5 Report Writing and Structure

- *"What testing and evaluation plans could you suggest for frontend, backend, database and ML."*

This back-and-forth helped shape the final structure and wording of the report, as well as the supporting diagrams and models.

Appendix D: Additional Code Samples

D.1 Session + Onboarding Management (Auth + localStorage)

```
// Restore session and accent colour on first load
useEffect(() => {
  const hasCompletedOnboarding = localStorage.getItem("onchord_onboarding_completed");
  const savedUserData = localStorage.getItem("onchord_user_data");
  const savedAccentColor = localStorage.getItem("onchord_accent_color");

  if (hasCompletedOnboarding === "true" && savedUserData) {
    try {
      const parsedUserData = JSON.parse(savedUserData);
      setUserData(parsedUserData);

      if (savedAccentColor) {
        setAccentColor(savedAccentColor);
      }

      setIsLoggedIn(true);
      setNeedsOnboarding(false);
    } catch (error) {
      console.error("Error  restoring user data:", error);
      // Start fresh if stored data is broken
      localStorage.removeItem("onchord_onboarding_completed");
      localStorage.removeItem("onchord_user_data");
      localStorage.removeItem("onchord_accent_color");
    }
  }
}, []);

// Handle login and decide whether to show onboarding
const handleLogin = (username: string, email: string) => {
  setUserData({ username, email });

  const hasCompletedOnboarding = localStorage.getItem("onchord_onboarding_completed");
```

```
  const savedUserData = localStorage.getItem("onchord_user_data");

  if (hasCompletedOnboarding === "true" && savedUserData) {
    try {
      const parsedUserData = JSON.parse(savedUserData);
      setUserData(parsedUserData);

      const savedAccentColor = localStorage.getItem("onchord_accent_color");
      if (savedAccentColor) {
        setAccentColor(savedAccentColor);
      }

      setIsLoggedIn(true);
      setNeedsOnboarding(false);
    } catch (error) {
      console.error("Error  restoring user data:", error);
      setNeedsOnboarding(true);
    }
  } else {
    // First-time user
    setNeedsOnboarding(true);
  }
};

const handleOnboardingComplete = (data: { displayName: string; accentColor: string }) => {
  setUserData(prev => ({ ...prev, username: data.displayName }));
  setAccentColor(data.accentColor);
  setNeedsOnboarding(false);
  setIsLoggedIn(true);

  // Persist onboarding state
  localStorage.setItem("onchord_onboarding_completed", "true");
  localStorage.setItem(
    "onchord_user_data",
    JSON.stringify({ username: data.displayName, email: userData.email })
```

```
  );
  localStorage.setItem("onchord_accent_color", data.accentColor);
};
```

This code shows how the prototype restores a logged-in session from localStorage, decides whether to show the onboarding flow, and saves the user's display name and accent colour once onboarding is complete.

D.2 Theme & Accessibility - Dark Mode + Accent Colour

```
// Toggle global dark mode class on the <html> element
useEffect(() => {
  if (darkMode) {
    document.documentElement.classList.add("dark");
  } else {
    document.documentElement.classList.remove("dark");
  }
}, [darkMode]);

// Apply the selected accent colour to CSS variables
useEffect(() => {
  const accentColors: Record<string, string> = {
    purple: "#A855F7",
    blue: "#3B82F6",
    pink: "#EC4899",
    green: "#10B981",
    orange: "#F59E0B",
  };

  const hex = accentColors[accentColor];
  document.documentElement.style.setProperty("--primary", hex);
  document.documentElement.style.setProperty("--accent", hex);
  document.documentElement.style.setProperty("--ring", hex);
}, [accentColor]);
```

These hooks keep the UI in sync with the user's theme choices by toggling a dark class and updating CSS variables for the accent colour.

D.3 Review Creation and Editing Flow

```
// Open the review modal pre-filled with album/song info
const handleOpenReviewModal = (
  mediaType: "album" | "song",
  mediaId: string,
  mediaTitle: string,
  mediaArtist: string,
  mediaCover: string
) => {
  setReviewMediaType(mediaType);
  setReviewMediaDetails({ id: mediaId, title: mediaTitle, artist: mediaArtist, cover:
mediaCover });
  setReviewModalOpen(true);
  setAlbumModalOpen(false);
};

// Create or update a review and persist it
const handleSubmitReview = (reviewData: {
  id?: string;
  rating: number;
  content: string;
  mood?: string;
  listeningContext?: string;
  favoriteTrack?: string;
  visibility: "public" | "friends" | "private";
}) => {
  if (reviewData.id) {
    // Edit existing review
    setAllReviews(prev => {
      const updated = prev.map(r =>
        r.id === reviewData.id
          ? {
              ...r,
```

```
            rating: reviewData.rating,
            content: reviewData.content,
            mood: reviewData.mood || "",
            listeningContext: reviewData.listeningContext || "",
            favoriteTrack: reviewData.favoriteTrack || "",
            visibility: reviewData.visibility,
            isEdited: true,
            editedAt: new Date().toISOString(),
          }
        : r
    );

    localStorage.setItem(
      "onchord_user_reviews",
      JSON.stringify(updated.filter(r => r.userId === "user-1"))
    );
    return updated;
  });
} else {
  // Create new review
  const newReview = {
    id: `r${Date.now()}`,
    userId: "user-1",
    userName: userData.username || currentUser.name,
    userAvatar: currentUser.avatar,
    albumId: reviewMediaDetails.id,
    albumTitle: reviewMediaDetails.title,
    albumArtist: reviewMediaDetails.artist,
    albumCover: reviewMediaDetails.cover,
    rating: reviewData.rating,
    type: reviewMediaType,
    content: reviewData.content,
    timestamp: "Just now",
    date: new Date().toISOString().split("T")[0],
    likes: 0,
```

```
      comments: 0,
      tags: [],
      mood: reviewData.mood || "",
      listeningContext: reviewData.listeningContext || "",
      favoriteTrack: reviewData.favoriteTrack || "",
      visibility: reviewData.visibility,
    };

    setAllReviews(prev => {
      const updated = [newReview, ...prev];
      localStorage.setItem(
        "onchord_user_reviews",
        JSON.stringify(updated.filter(r => r.userId === "user-1"))
      );
      return updated;
    });
  }
};
```

This handler starts the “Write Review” flow. It either updates an existing review or creates a new one, updates React state and saves the user’s own reviews into localStorage so they continue between sessions.

Appendix E: Additional project analysis

1. Letterboxd (https://letterboxd.com)

2. Last.fm (https://www.last.fm)

3. Stats.fm (formerly Spotistats) (https://stats.fm)

4. Discogs (https://www.discogs.com)

5. RateYourMusic (RYM) (https://rateyourmusic.com)

6. Bandcamp (https://bandcamp.com)

7. Goodreads (https://www.goodreads.com)

8. SoundCloud (https://soundcloud.com)

9. Spotify (Social Features) (https://spotify.com)

10. Obscurify (https://www.obscurifymusic.com/login)

Appendix F: Technology Research References

Frontend Technologies

- **React** - https://react.dev
- **TypeScript** - https://www.typescriptlang.org
- **Tailwind CSS** - https://tailwindcss.com
- **shadcn/ui** - https://ui.shadcn.com
- **Radix UI** - https://www.radix-ui.com

Backend & Database

- **Node.js** - https://nodejs.org
- **Express** - https://expressjs.com
- **Supabase** - https://supabase.com
- **MongoDB Atlas** (if used) - https://www.mongodb.com/atlas

Machine Learning Stack

- **Python** - https://www.python.org
- **FastAPI** - https://fastapi.tiangolo.com
- **scikit-learn** - https://scikit-learn.org
- **pandas** - https://pandas.pydata.org
- **NumPy** - https://numpy.org

External APIs

G) Spotify Web API

https://developer.spotify.com/documentation/web-api

H) Apple Music API

https://developer.apple.com/documentation/applemusicapi/

I) Ticketmaster Discovery API

https://developer.ticketmaster.com/products-and-docs/apis/discovery-api/v2/

Appendix G: Other Use Case Narratives

USE CASE	UC-04	**View Insights & Discover Music (Stats, Taste Match, Recommendations)**
Description of Goal in Context	The user wants to explore personalised listening insights, music analytics, and discovery features based on their listening habits. The goal is to provide meaningful reflections on the user's music identity and present AI-powered recommendations and taste matches.	
Preconditions	• User is logged in. • User has linked a Spotify account.	
Post Conditions, Success End Condition	• A set of insights and recommended music is displayed. • No user data is changed.	
DESCRIPTION of Scenario	The user navigates to the Insights page. The system analyses the user's listening history and preferences, then presents summaries (e.g. top artists, genres, moods) and suggested music to explore.	

Main Flow

Step	**Action**	**Alternate**
n.1	User selects the **Insights** tab	
n.2	System gathers the user's listening history and profile data	
n.3	System analyses this data (e.g. patterns, similarities, moods)	
n.4	System displays charts, summaries, and recommended music	

EXCEPTIONS or ERROR Flow

EF1 - Insights Cannot Be Generated

Step	**Branching Action**	**Alternate**
n.m.1	Listening data is missing or cannot be analysed	
n.m.2	System shows a message ("Insights currently unavailable") and, where possible, displays basic stats only	

ALTERNATIVE or VARIATION Flow

Description

AF1 - Previously Generated Insights

Step	**Branching Action**	**Alternate**
n.m.1	Fresh analysis is slow or unavailable	

n.m.2	System shows insights generated earlier and marks them as previously calculated	

USE CASE	UC-05	**View Public Content (Reviews, Playlists, Favorites)**
Description of Goal in Context	A friend user wants to view another user's publicly visible reviews, playlists, and favourites to discover new music and understand their taste	
Preconditions	• Viewer is logged in. • Public content exists for the target user.	
Post Conditions, Success End Condition	• Public content is displayed to the viewer. • No data changed.	
DESCRIPTION of Scenario	The friend user opens another user's profile. The system shows all content that has been marked as public or visible to friends.	

Main Flow		
Step	**Action**	**Alternate**
n.1	Friend User selects another user's profile	
n.2	System loads public reviews, playlists, and favourites	
n.3	System displays content in profile tabs	
n.4	Friend user browses the available content	
EXCEPTIONS or ERROR Flow **Description** **EF1 - No Public Content Available**		
Step	**Branching Action**	**Alternate**
n.m.1	Target user has no public content or has a private profile	
n.m.2	System displays: "No public content available."	
ALTERNATIVE or VARIATION Flow **Description** AF1 - **Friends-Only Content**		
Step	**Branching Action**	**Alternate**
n.m.1	Viewer is on the target user's friends list	

n.m.2	System also shows content marked “Friends Only”	

USE CASE	UC-06	**Interact Socially (Likes, Comments, Shares)**
Description of Goal in Context	The user interacts with content (e.g. reviews, playlists) by liking, commenting, or sharing, to participate in discussion and show appreciation.	
Preconditions	• User is logged in. • Target content exists and is public.	
Post Conditions, Success End Condition	• The like/comment/share is recorded. • Interaction counts and comment threads are updated in the interface.	
DESCRIPTION of Scenario	While viewing public or friend content, the user can like it, comment on it, or share it to their own space. The system updates the visible counts and comment list.	

Main Flow		
Step	**Action**	**Alternate**
n.1	User opens a content item	
n.2	System shows interaction options	
n.3	User selects one or more actions	
n.4	System saves the chosen interactions	
n.5	System updates the counts and displays any new comments	
EXCEPTIONS or ERROR Flow **Description** **EF1 - Interaction Save Fails**		
Step	**Branching Action**	**Alternate**
n.m.1	System cannot record the interaction	
n.m.2	System shows “Unable to process interaction.”	
ALTERNATIVE or VARIATION Flow **Description** AF1 - **Commenting**		
Step	**Branching Action**	**Alternate**
n.m.1	User selects **Comment**	

n.m.2	System opens a comment editor	
n.m.3	User writes and submits the comment	
n.m.4	System adds the comment to the thread	

USE CASE	UC-07	**Follow / Unfollow User**
Description of Goal in Context	The user chooses to follow or unfollow another user to customise their social feed and build music-based connections.	
Preconditions	• Both users exist on the platform. • The acting user is logged in.	
Post Conditions, Success End Condition	• The follow relationship is added or removed. • Follower/following counts and follow button state are updated.	
DESCRIPTION of Scenario	The user opens another user's profile and presses the follow or unfollow button. The system updates their connection and reflects the change on screen.	

Main Flow

Step	**Action**	**Alternate**
n.1	User opens another user's profile	
n.2	System loads follow status	
n.3	User selects **Follow** or **Unfollow**	
n.4	System updates the follow relationship	
n.5	System refreshes counts and button label	

EXCEPTIONS or ERROR Flow

Description

EF1 - Follow/Unfollow Fails

Step	**Branching Action**	**Alternate**
n.m.1	System cannot complete the follow/unfollow action	
n.m.2	System displays an error message	

ALTERNATIVE or VARIATION Flow

Description

AF1 - **Follow Back Notification**

Step	**Branching Action**	**Alternate**
n.m.1	User views someone who already follows them	

n.m.2	System highlights this with a “Follows you” / “Follow back” indicator	